

TRABAJO FIN DE GRADO

INGENIERÍA INFORMÁTICA

Aplicación para la gestión del hogar en piso de estudiantes

Autor

Sergio Muñoz Gómez

Directores

María Luisa Rodríguez Almendros



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

—

Granada, Enero de 2025

Aplicación para la gestión del hogar en piso de estudiantes

Sergio Muñoz Gómez

Palabras clave: palabra_clave1, palabra_clave2, palabra_clave3, ...

Resumen

Poner aquí el resumen.

Project Title: Project Subtitle

First name, Family name (student)

Keywords: Keyword1, Keyword2, Keyword3, ...

Abstract

Write here the abstract in English.

Yo, **Sergio Muñoz Gómez**, alumno de la titulación INGENIERÍA INFORMÁTICA de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 76062609E, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Sergio Muñoz Gómez

Granada a 20 de Enero de 2025 .

D. **María Luisa Rodríguez Almendros**, Profesora del Área de Ingeniería del Software del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***Aplicación para la gestión del hogar en piso de estudiantes***, ha sido realizado bajo su supervisión por **Sergio Muñoz Gómez**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 20 de Enero de 2025 .

Los directores:

María Luisa Rodríguez Almendros

Agradecimientos

Poner aquí agradecimientos...

Índice general

1. Introducción **17**

1.1. Motivación 17

1.2. Objetivos del proyecto 18

1.3. Estructura de la memoria 19

2. Planificación y presupuesto **21**

2.1. Fases 21

2.1.1. Fase Inicial 21

2.1.2. Fase de Investigación 21

2.1.3. Fase de Analisis 22

2.1.4. Fase de Diseño 22

2.1.5. Fase de Desarrollo 22

2.1.6. Documentación 22

2.1.7. Diagrama de Gantt 22

2.2. Presupuesto 24

2.2.1. Recursos Hardware 24

2.2.2. Recursos Software 24

2.2.3. Licencias 25

2.2.4. Costes indirectos 25

2.2.5. Presupuesto Total 25

3. Estado del arte **27**

3.1. Estudios sobre la gestión del hogar y las finanzas en viviendas compartidas 27

3.2. Aplicaciones similares y comparativas 30

3.2.1. Tricount 30

3.2.2. Todoist 34

3.2.3. MeToca 39

3.2.4. Cozi Family Organizer 40

3.2.5. OurHome 45

3.2.6. Comparativa 46

3.3. Legislación actual de protección de datos 48

3.4. Estudio de usabilidad y accesibilidad 50

3.4.1. Estudio de usabilidad 50

3.4.2. Estudio de accesibilidad 50

4. Análisis de tecnologías **52**

4.1. Análisis de las tecnologías Frontend 52

4.1.1. Desarrollo nativo 52

4.1.2. Desarrollo Híbrido 53

4.1.3. Tabla comparativa de tecnologías front-end 55

4.2. Analisis de las tecnologías Backend 57

4.2.1. Firebase 57

4.2.2. Node.js con Express 57

4.2.3. Django 58

4.2.4. Ruby on Rails 58

4.2.5. Tabla comparativa de tecnologías back-end 59
4.3. Justificación de la elección de tecnologías 60

5. Análisis de metodologías de desarrollo 61
5.1. Metodologías de desarrollo de software 61
5.1.1. Modelo en cascada (Waterfall) 61
5.1.2. Modelo en espiral 62
5.1.3. Metodología iterativa 62
5.1.4. Desarrollo iterativo e Incremental 62
5.1.5. Metodología Ágil 63
5.1.6. DevOps 63
5.2. Comparativa de metodologías 63
5.3. Justificación de la metodología 64

6. Análisis de la aplicación 65
6.1. Especificación de requisitos 65
6.1.1. Requisitos funcionales 65
6.1.2. Requisitos no funcionales 88
6.2. Modelos de casos de uso 89
6.2.1. Diagrama de casos de uso 89
6.2.2. Descripción de los casos de uso 93

7. Diseño 96
7.1. Diseño de la arquitectura 96
7.2. Diseño de la interfaz gráfica 96
7.2.1. Interfaces de gráficas del Grupo 1: *Gestión de sesión* . 96
7.2.2. Interfaces de gráficas del Grupo 2: *Gestión del hogar* . 100

7.2.3. Interfaces de gráficas del Grupo 3: *Gestión de tareas* . 102

7.2.4. Interfaces de gráficas del Grupo 4: *Gestión de gastos* . 103

7.2.5. Interfaces de gráficas del Grupo 5: *Gestión de la compra* 103

A. Descripción de casos de uso **104**

Referencias 111

Índice de figuras

3.1. Número de personas que residen en la vivienda. 27
3.2. Pantalla Inicio. 31
3.3. Lista de grupos. 31
3.4. Añadir un nuevo grupo. 32
3.5. Pantalla del perfil. 32
3.6. Detalles de un nuevo grupo. 33
3.7. Enviar invitación a los miembros del grupo. 33
3.8. Añadir un gasto. 33
3.9. Resumen de los gastos y deudas. 33
3.10. Pantalla de registro. 36
3.11. Elegir el ámbito de la aplicación. 36
3.12. Pantalla principal. 37
3.13. Añadir una tarea. 37
3.14. Pantalla con la tarea añadida. 38
3.15. Calendario de tareas. 38
3.16. Registro. 42
3.17. Crear una familia. 42
3.18. Añadir un evento. 43

3.19. Calendario. 43
3.20. Listas de la compra. 44
3.21. Recetas. 44

4.1. Logo de Kotlin 53
4.2. Logo de Swift 53
4.3. Logo de Ionic 54
4.4. Logo de Flutter 54
4.5. Logo de React Native 55
4.6. Logo de Xamarin 55
4.7. Logo de Firebase 57
4.8. Logo de Node.js 58
4.9. Logo de Django 58
4.10. Logo de Ruby on Rails 59

6.1. Diagrama de paquetes de casos de uso 89
6.2. Diagrama de casos de uso de la gestión de sesión 90
6.3. Diagrama de casos de uso de la gestion de hogar 90
6.4. Diagrama de casos de uso de la gestion de tareas 91
6.5. Diagrama de casos de uso de la gestion de gastos 91
6.6. Diagrama de casos de uso de la gestion de compra 92

7.1. Diseño de la pantalla de bienvenida. 97
7.2. Diseño de la pantalla de inicio de sesión. 97
7.3. Diseño de la pantalla de registro. 98
7.4. Diseño de la pantalla de cierre de sesión y datos de usuario. . 98
7.5. Diseño de la pantalla de editar perfil. 99

7.6. Diseño de la pantalla para unirte a un hogar o crear uno nuevo. 100
7.7. Diseño de la pantalla para unirte a un hogar. 100
7.8. Diseño de la pantalla para crear un nuevo hogar. 101
7.9. Diseño de la pantalla para abandonar a un hogar. 101
7.10. Diseño de la pantalla principal de tareas. 102
7.11. Diseño de la pantalla para añadir una tarea. 102

Índice de cuadros

2.1. Tabla de tareas y horas 23

2.2. Presupuesto detallado del proyecto 26

3.1. Comparativa de aplicaciones para la gestión de tareas y gastos. 46

4.1. Comparativa de tecnologías para desarrollo de aplicaciones móviles. 56

4.2. Comparativa de tecnologías para el desarrollo back-end. . . . 59

5.1. Comparativa de metodologías de desarrollo de software. . . . 63

6.1. RF_01 66

6.2. RF_02 67

6.3. RF_03 68

6.4. RF_04 68

6.5. RF_05 69

6.6. RF_06 70

6.7. RF_07 71

6.8. RF_08 72

6.9. RF_09 72

6.10. RF_10 73

6.11. RF_11 74
6.12. RF_12 75
6.13. RF_13 76
6.14. RF_14 76
6.15. RF_15 77
6.16. RF_16 77
6.17. RF_17 78
6.18. RF_18 78
6.19. RF_19 79
6.20. RF_20 80
6.21. RF_21 80
6.22. RF_22 81
6.23. RF_23 81
6.24. RF_24 82
6.25. RF_25 83
6.26. RF_26 84
6.27. RF_27 85
6.28. RF_28 85
6.29. RF_29 86
6.30. RF_30 87
6.31. RNF_01 88
6.32. RNF_02 88
6.33. RNF_03 88
6.34. RNF_04 88
6.35. RNF_05 88

6.36. Caso de uso - Registro 93
6.37. Caso de uso - Iniciar Sesión 94
6.38. Caso de uso - Cerrar Sesión 95

A.1. Caso de uso - Registro 105
A.2. Caso de uso - Iniciar Sesión 106
A.3. Caso de uso - Cerrar Sesión 107
A.4. Caso de uso - Ver Usuario 108
A.5. Caso de uso - Editar Usuario 109
A.6. Caso de uso - Eliminar Usuario 110

Capítulo 1

Introducción

1.1. Motivación

La mayoría de estudiantes enfrentan dificultades para gestionar adecuadamente su hogar debido a la falta de tiempo, ya que gran parte de su jornada está dedicada a sus estudios. Esta situación convierte la organización y la realización de tareas domésticas en un problema, pues estas actividades requieren una planificación cuidadosa y tiempo para llevarlas a cabo. Por otro lado, la gestión de los gastos compartidos puede resultar especialmente compleja en un entorno de estudiantes, ya que es común que cada miembro del hogar se encargue de diferentes pagos. Esto puede generar confusiones y equivocaciones, respectoa a quién ha pagado ciertos gastos y qué cantidades están pedientes, lo que agrava aún más la dificultad de una correcta administación del hogar.

Después de 5 años viviendo de manera independiente, he experimentado de primera mano las dificultades de gestionar un hogar compartido con otros estudiantes o trabajadores. En numerosas ocasiones el tiempo libre del que disponíamos no se ha destinado a la organización del hogar. Aunque las tareas se asignaban semanalmente, no siempre se cumplían, ya sea por falta de tiempo y o por descuidos. Además, la administación de los gastos ha sido, en muchas ocasiones, caótica, ya que no siempre se llevaba un control adecuado sobre quién ha pagado qué, y algunas personas solían olvidar devolver el dinero a tiempo.

En la actualidad, aproximadamente el **87 % de los jovenes emancipados en España viven comparten vivienda**, segun un estudio del portal HuffPost. (Huffington Post, 2025). Además, el 31 % de quienes alquilan una habitación en un piso compartido cohabitan con otras tres personas. (FC Research, 2024). Por tanto, la necesidad de una aplicación que facilite la organización y gestión del hogar en pisos de estudiantes es evidente.

Actualmente no existe ninguna aplicación que ofrezca una solución integral tanto para la gestión de las tareas domesticas como para la gestión de los gastos. Las aplicaciones disponibles abordan estos aspectos por separado. Además la mayoría de ellas están orientadas a hogares familiares donde la distribución de las tareas no se gestiona de la misma forma que en los pisos de estudiantes. Por eso es fundamental desarrollar una aplicación que integre todas estas funcionalidades, dirigida específicamente a estudiantes o trabajadores que compartan vivienda.

La aplicación que quiero desarrollar estará diseñada para ayudar a estudiantes y trabajadores que compartan vivienda, proporcionando una solución integrada que facilite tanto la organización de las tareas domésticas como la administración de gastos compartidos. En un entorno de convivencia, la asignación de tareas suele complicarse debido a los horarios irregulares y las diferentes cargas de estudio o trabajo de cada persona. Esta aplicación permitirá a asignar tareas de manera equitativa y flexible, estableciendo recordatorios y notificando a cada miembro sobre las responsabilidades asignadas para mejorar el cumplimiento y la colaboración en el hogar.

Por otra parte, la aplicación incluirá un sistema de gestión de gastos compartidos, donde se registrarán todos los pagos realizados y las cantidades que se deben por cada persona. Esto reducirá las confusiones y facilitará un control transparente y detallado de los gastos del hogar. En lugar de depender de otras aplicaciones o de anotaciones manuales, esta herramienta unificará ambos aspectos (tareas y gastos) en un solo lugar, simplificando la administración y ayudando a la organización en hogares compartidos.

1.2. Objetivos del proyecto

El objetivo principal del proyecto es desarrollar una aplicación móvil para facilitar a los estudiantes la organización y gestión eficiente de su hogar, ofreciendo soluciones que aborden las dificultades que puedan surgir en la planificación y distribución de responsabilidades. Para poder alcanzar este objetivo, podemos diferenciar una serie de objetivos específicos:

- **Objetivo 1: Análisis de la gestión de hogares compartidos y gestión de gastos.**
  - **Descripción:** Búsqueda de información sobre los problemas más comunes que surgen en la gestión de hogares compartidos y en la gestión de gastos.
- **Objetivo 2: Estudio de aplicaciones similares**

    - **Descripción:** Estudio del mercado actual de aplicaciones similares con el fin de identificar funcionalidades innovadoras que puedan ser incorporadas en nuestra aplicación.

- **Objetivo 3: Análisis de las tecnologías a utilizar**
    - **Descripción:** Investigar cuáles son las tecnologías más adecuadas para la implementación del proyecto.
- **Objetivo 4: Desarrollo e implementación de la aplicación**
    - **Descripción:** Analizar, desarrollar y realizar pruebas en la aplicación con el fin de obtener los mejores resultados.
- **Objetivo 5: Análisis de usabilidad y accesibilidad**
    - **Descripción:** Realizar un análisis de usabilidad y accesibilidad para identificar posibles mejoras en la interfaz de usuario.
- **Objetivo 6: Validación de la aplicación por usuarios reales**
    - **Descripción:** Evaluar la aplicación en un entorno real para verificar si satisface las expectativas del usuario.

1.3. Estructura de la memoria

A continuación, se ofrece una visión general de la estructura y organización de la memoria. El contenido se ha dividido en los siguientes apartados:

1. **Introducción.** En este apartado se presenta la motivación del proyecto y los objetivos que se pretenden alcanzar.
2. **Planificación y presupuesto.** Descripción de la planificación del proyecto, que incluye las diferentes fases que se han llevado a cabo durante su desarrollo, así como el presupuesto estimado.
3. **Estado del arte.** Estudio sobre los principales temas relacionados con este proyecto, cómo analisis de aplicaciones similares, temas legales de protección de datos, etc.
4. **Análisis de tecnologías.** Análisis de las posibles tecnologías que se pueden usar en la actualidad y eleeciión de las más adecuadas para el desarrollo de la aplicación.
5. **Análisis de metodologías de desarrollo** Análisis de las posibles metodologías que existen y elección de las más adecuadas para el desarrollo de la aplicación.

6. **Análisis de la aplicación.** Se detalla el análisis de la aplicación, incluyendo los requisitos funcionales y no funcionales, así como los casos de uso.

7. **Diseño.** Descripción del diseño de la aplicación, incluyendo la arquitectura, el diseño de la base de datos y la interfaz de usuario.

8. **Desarrollo e implementación de la aplicación.** Se detalla el proceso de desarrollo de la aplicación. Se realizará siguiendo una metodología iterativa y se detallarán sus iteraciones.

9. **Validación de la aplicación por usuarios reales.** Evaluación de la aplicación en un entorno real para verificar si satisface las expectativas del usuario.

10. **Conclusiones y trabajo futuro.** Se presentan las conclusiones obtenidas a lo largo del proyecto y se proponen posibles líneas de trabajo futuro.

11. **Bibliografía.** Referencias bibliográficas utilizadas en la elaboración de la memoria.

Capítulo 2

Planificación y presupuesto

2.1. Fases

La planificación del proyecto se estructurará en fases con el objetivo de garantizar un desarrollo organizado y eficiente, asegurando que cada etapa esté enfocada en el cumplimiento de objetivos específicos.

2.1.1. Fase Inicial

En esta fase se recopilará y definirá toda la información relacionada con el Capitulo 1, incluyendo la motivación para la realización del proyecto y la determinación de los objetivos que se pretenden alcanzar a lo largo del mismo.

2.1.2. Fase de Investigación

Esta fase estará dedicada a la recopilación de información relevante para el desarrollo del proyecto. Se analizarán aplicaciones similares para demostrar la necesidad de una aplicación integral que cubra todas las funciones propuestas. Además se investigará aspectos como la gestión del tiempo en hogares compartidos y la administración de gastos entre personas que comparten un mismo domicilio.

Para garantizar que la aplicación cumpla con los requisitos legales, también se revisará la normativa vigente de protección de datos. Asímismo, se realizará un estudio de usabilidad y accesibilidad para asegurar que el diseño sea inclusivo y fácil de usar para todos los usuarios.

2.1.3. Fase de Analisis

En esta fase se llevará a cabo un análisis de las diferentes tecnologías que se pueden utilizar para el desarrollo de la aplicación, tanto para el frontend como para el backend, con el objetivo de selecionar las más adecuadas para el proyecto. Además, se definirán los requisitos que debe cumplir la aplicación para satisfacer las necesidades del usuario objetivo. Como parte de esta fase, se elaborarán los casos de uso y los diagramas de secuencia del sistema.

2.1.4. Fase de Diseño

Esta fase estará enfocada en la creación de los prototipos de la aplicación. Se desarrollará un diagrama de arquitectura del sistema y bocetos para la interfaz gráfica. Además, se definirán los colores y las tipografías que se emplearán en la aplicación, garantizando un diseño responsive que permita su visualización en cualquier dispositivo.

2.1.5. Fase de Desarrollo

En esta fase se procederá a la implementación de la aplicación, siguiendo las tecnologías, metodologías y diseños definidos en las fases previas de análisis y diseño. Esta fase se dividirá en cuatro iteraciones, ya que se empleará una metodología ágil para el desarrollo del proyecto. En cada iteración se determinarán las funcionalidades que se implementarán y se realizarán pruebas para garantizar que la aplicación cumple con los requisitos definidos en la fase de análisis.

Las pruebas serán realizadas por usuarios reales, concretamente jóvenes que comparten piso, para verificar que la aplicación satisface sus necesidades y resulta fácil de utilizar.

2.1.6. Documentación

Aunque no se considera una fase independiente, la documentación será un proceso continuo a lo largo de todo el proyecto. Se registrarán de manera detallada todas las actividades y resultados obtenidos en cada fase.

2.1.7. Diagrama de Gantt

En esta tabla se específica la duración en horas de cada una de las tareas representadas en el diagrama de Gantt. Este diagrama representa la planificación temporal de las actividades del proyecto, donde cada celda corresponde a una semana del mes. La tabla presenta la cantidad de horas

dedicadas a cada tarea, mientras que el diagrama de Gantt muestra el periodo en el que se llevaron a cabo.

Tarea	**Horas**
Motivación	-
Objetivos del proyecto	-
Estructura de la memoria	-
Planificación	✓
Presupuesto	–
Aplicaciones similares y comparativas	–
Estudio de la distribución del tiempo y gestión del hogar en entornos compartidos	–
Estudio sobre los desafíos financieros en viviendas estudiantiles compartidas	–
Legislación actual de protección de datos	–
Estudio de usabilidad y accesibilidad	–
Análisis de las tecnologías Frontend	–
Analisis de las tecnologías Backend	–
Justificación de la elección de las tecnologías	–

Cuadro 2.1: Tabla de tareas y horas

2.2. Presupuesto

En este apartado se detallarán los costes asociados al desarrollo del proyecto. Se incluirán los costes de hardware, software, licencias, servicios en la nube y otros gastos necesarios para la realización del proyecto. Realizar un presupuesto en este proyecto es clave para planificar y gestionar los recursos de forma eficiente. Permite asegurarse de que las fases de desarrollo se puedan llevar a cabo con los medios disponibles. Además, ayuda a justificar las decisiones tomadas y evitar problemas por faltas de recursos asegurando que el proyecto sea viable y esté bien estructurado.

2.2.1. Recursos Hardware

Los componentes hardware necesarios para el desarrollo del proyecto son un ordenador portátil y un teléfono móvil para realizar pruebas con la aplicación.

- **Ordenador:** TUF Gaming FX505DT_FX505DT
- **Teléfono Móvil:** Samsung Galaxy A51

2.2.2. Recursos Software

Las herramientas software que se utilizarán en el proyecto son las siguientes:

- **Sistema Operativo:** Windows 11
- **IDE:** Visual Studio Code
- **Control de Versiones:** Git
- **Lenguajes de Programación:** Flutter, Dart
- **Diseño de diagramas UML:** Visual Paradigm
- **Diseño de Interfaces:** Figma
- **Sistema de Gestión de Base de Datos:** Firebase
- **Sistema de composición de texto:** LaTeX

2.2.2.1. Recursos Humanos

En este apartado se detallarán los gastos por la contratación de personal. Este proyecto será desarrollado por una única persona, un programador junior, que según (Talent.com, 2025) tiene un coste de 21.000€al año, como la duración de este proyecto es de 6 meses, el coste total será de 10.500€.

2.2.3. Licencias

En este apartado se detallan las licencias software necesarias para llevar a cabo el desarrollo del proyecto. Aunque en este caso no supone ningún coste, ya que se utilizarán versiones gratuitas o herramientas sin coste, en un proyecto real sería imprescindible considerar el pago de las licencias. Esto garantizaría el acceso completo a todas las funcionalidades de las herramientas, además de cumplir con las normativas legales y éticas relaciondas con el uso del sotware. Las licencias de software que deberían considerarse en un proyecto real son las siguientes:

- **Windows 11:** Windows 11 Home License
- **Visual Paradigm:** Visual Paradigm Standard Edition
- **Figma:** Figma Professional

El resto de licencias necesarias para el desarrollo del proyecto son gratuitas, por lo que no se incluirán en el presupuesto.

2.2.4. Costes indirectos

En este apartado se engloba todos los gastos indirectos que incluyen los costes de electricidad, internet, espacio de trabajo, etc. Estos gastos indirectos suelen estimarse como un porcentaje adicional del coste directo total *(entre 10 % y 20 %)*. En este caso será un 15 % de los recursos humanos.

2.2.5. Presupuesto Total

El coste total del proyecto se obtiene sumando los costes de los recursos hardware, software, humanos, licencias y costes indirectos.

Concepto	Coste Mensual	Coste Total
Recursos Hardware	-	1.000 €
Ordenador portátil	-	800 €
Teléfono móvil para pruebas	-	200 €
Recursos Humanos	1.750€	10.500 €
Programador Junior (6 meses)	1.750€	10.500 €
Licencias	52,68€	316,12€
Windows 11 Home License	-	145 €
Visual Paradigm Standard Edition	12,23€	73,38€
Figma Professional	20,45€	122,7€
Costes Indirectos	262,5€	1.575€
Electricidad	50€	300€
Internet	30€	180€
Espacio de trabajo (estimado)	182,5€	1.095€
Coste Total	2.065,18€	**13.391,12€**

Cuadro 2.2: Presupuesto detallado del proyecto

Capítulo 3

Estado del arte

3.1. Estudios sobre la gestión del hogar y las finanzas en viviendas compartidas

Según el informe "Un problema como una casa" del consejo de la Juventud de España (CJE) el **87 % de los jovenes emancipados en España comparten vivienda** (Huffington Post, 2025). Esta tendencia se debe a la necesidad de reducir los gastos, ya que los altos costos de alquiler y la precariedad laboral dificultan que los jóvenes puedan afrontar los gastos mensuales de una vivienda en solitario. Como podemos ver en la grafica de la imagen 3.1 dentro de los jóvenes emancipados que comparten vivienda, **alrededor del 42,4 % la comparten con 3 o 4 personas** y el 37,9 % con dos personas más, siendo el restante personas que vivien con una persona más o con más de cuatro.

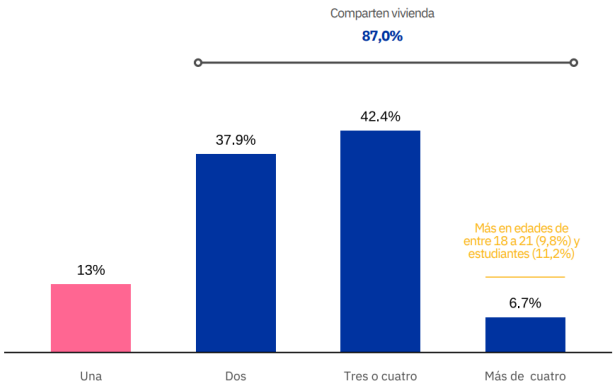


Figura 3.1: Número de personas que residen en la vivienda.

Para hacer frente a esta situación, muchos jóvenes recurren al apoyo económico familiar: alrededor del 30 % necesita ayuda de sus familias para cubrir los gastos mensuales. Esta dependencia económica refleja la insuficiencia de los ingresos juveniles para sostener una vida independiente.

La calidad de las viviendas compartidas también es motivo de preocupación. Casi la mitad de los jóvenes que viven de alquiler deben afrontar una fianza equivalente a dos meses de renta, y un 41,5 % paga más de 600 euros en este concepto. Además, el 50 % ha tenido problema con sus caseros, principalmente con la falta de reparación de desperfectos.

Un estudio del *American Time Use Survey*, (American Time Use Survey, 2022) analiza la distribución del tiempo que las personas dedican a la administración del hogar y cómo esta tarea se realiza en intervalos dispersos. Este estudio revela patrones en las actividades domésticas y ayuda a entender los desafío de coordinación y planificación en hogares donde las responsabilidades se dividen entre varias personas. El estudio muestra que las tareas del hogar, como la limpieza, la preparación de comidas, la organización y la gestión de finanzas, tienden a realizarse en intervalos de tiempo que varían significativamente entre los distintos miembros del hogar. Esto es problemático en hogares compartidos, como los estudiantes o trabajadores, donde la disponibilidad de cada persona depende de su carga académica o laboral. La falta de tiempos fijos y la irregularidad en la dedicación a las tareas puede generar problemas de coordinación, ya que cada miembro tiene su propio horario, lo que impide una rutina doméstica unificada.

Otro hallazgo importante de este estudio es que las tareas domésticas se realizan de manera inconsistente debido a limitaciones de tiempo y energía, especialmente entre aquellos con empleos de tiempo completo o estudiante con cargas académicas pesadas. Esta incosistencia es común en los hogares compartidos, donde la carga se divide entre varias personas sin un sistema claro de asignación de responsabilidades o seguimiento. Sin herramientas adecuadas para la organización, es posible que algunas tareas queden sin realizar o que recaigan siempre en las mismas personas, generando frustración y posibles conflictos.

El estudio también resalta que, a medida que los individuos se ven cada vez más presionados por sus estudios o trabajos, tienden a posponer las tareas del hogar, lo cual se agrava en entornos de convivencia compartida. Este aplazamiento puede llevar a una acumulación de tareas, que a su vez aumenta el nivel de estrés de los residentes y genera tensiones en el hogar.

Por último, los resultados también sugieren que la organización efectiva del hogar es crucial para el bienestar de sus miembros. En el contexto de hogares compartidos, el uso de herramientas de planificación o aplicaciones específicas podría mejorar significativamente la administración de tareas y reducir el estrés relacionado con el hogar.

El estudio basado en datos de HUD, (U.S. Department of Housing and Urban Development, 2022) examina los desafíos financieros que enfrentan los estudiantes en viviendas compartidas. Uno de los hallazgos principales de este estudio es que los contratos de arrendamiento de las viviendas a menudo requieren compromisos anuales. Los estudiantes deben seguir pagando el alquiler durante todo el año, incluso en períodos en que la vivienda está vacía, como los meses de verano. Este requisito genera una carga financiera adicional, ya que los estudiantes deben encontrar formas de cubrir los pagos durante esos meses.

También señala que la organización de los gastos compartidos es una tarea compleja para los estudiantes. Dado que los contratos incluyen puede incluir diferentes servicios, la variación en las facturas mensuales añade dificultad al reparto equitativo de costes. Muchos estudiantes carecen de herramientas y sistemas de organización, lo que provoca confusión y conflictos en cuanto a las cantidades que cada miembro debe pagar. Esto puede ser díficil de gestionar cuando uno o varios de los residentes enfrentan problemas financieros, ya que retrasos o impagos afectan a todos los miembros del hogar.

Por último, este estudio también examina cómo las dificultades para gestionar los gastos afectan no solo la situación financiera de los estudiantes, sino también su bienestar general y rendimiento académico. La preocupación por la estabilidad financiera y la tensión asociada con la administración de los costos compartidos afectan a los estudiantes a nivel psicológico y académico, ya que deben dedicar su tiempo y energía a resolver estos problemas, lo que limita su concentración en los estudios.

3.2. Aplicaciones similares y comparativas

3.2.1. Tricount

Tricount es una aplicación diseñada para facilitar la gestión y el reparto de gastos entre varias personas. Es útil en sistuaciones, donde un grupo de personas realiza gastos comunes, como en viajes o cenas. Es una aplicación bastante sencilla de usar, lo que la hace aún más popular entre los grupos de personas. Está disponible para Android e iOS y tiene también soporte web. (Tricount, 2025)

3.2.1.1. Funciones Principales

1. **Creación de un Tricount**
   - *a*) Configuración Inicial. Al iniciar un nuevo "Tricount"(grupo) se le da un nombre, se elige un tipo de divisa y se añade a los participantes.
   - *b*) Acceso Compartido. Puede ser compartido entre todos los participantes a través de un enlace, lo que permite que cada uno pueda añadir sus gastos.
2. **Añadir Gastos**
   - *a*) Registro de Gastos. Cualquier participante puede añadir un gasto en el tricount, especificando la cantidad, la categoría, quien pagó y para quien es ese gasto.
   - *b*) División Personalizada. Tricount permite dividir los gastos de forma equitatica o personalizada, es decir, si el gasto es para todos o solo para algunos.
   - *c*) Comentarios. Se pueden añadir comentarios a cada pago.
3. **Cálculo Automático**
   - *a*) Resumen de Balances. Se calcula automáticamente cúanto debe cada persona y a quien.
   - *b*) Balances Individuales. Cada participante puede ver su valance individual y cuánto ha gastado, cuánto le deben y cuánto le debe a otros.
4. **Saldar Deudas**
   - *a*) Recomendaciones de Pagos. Tricount recomienda la mejor manera de saldar las deudas dentro del grupo, optimizando el número de transsaciones necesarias.

b) Pagos Parciales o Totales. Los participantes pueden registrar pagos parciales o totales dentro de la aplicación para mantener un seguimiento claro de que ha sido pagado y lo que queda pendiente.

3.2.1.2. Modo de Uso

Una vez nos hayamos descargado la aplicación ya sea desde Google Play o App Store, lo primero que nos muestra es una pantalla de inicio como nos muestra la *Figura 3.2*.

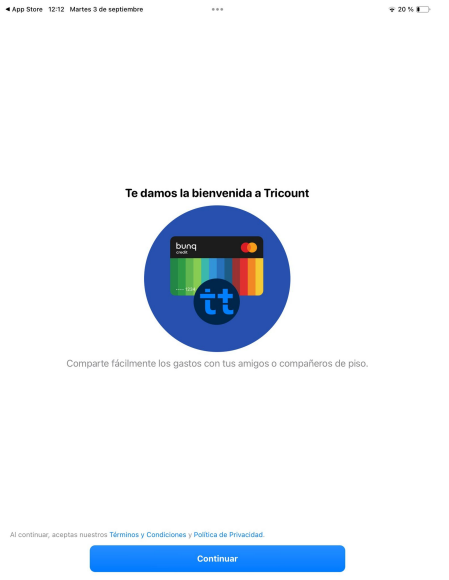


Figura 3.2: Pantalla Inicio.

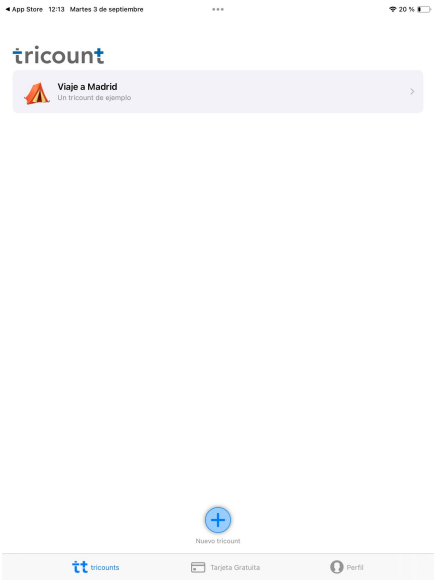


Figura 3.3: Lista de grupos.

Pulsado el botón de *Continuar* nos muestra una pantalla donde nos sale una lista de los grupos que hemos creado o en los que estamos participando. En este caso como es la primera vez que entramos solo nos muestra un grupo de ejemplo (*Figura 3.3*).

Para añadir un nuevo grupo, pulsamos el botón de *Nuevo Tricount* y nos muestra una pantalla emergente donde nos da dos opciones, la primera es iniciar un Tricount (grupo) desde 0 y la otra es unirnos a un grupo ya existente (*Figura 3.4*).

Otra de las pestañas que nos muestra es la del perfil, donde nos permite iniciar sesión y en caso de que ya estuvisemos registrado, se nos importan todos los grupos que estuvieran asociados a nuestra cuenta (*Figura 3.5*). Ahora vamos a hablar de lo realmente importante de la aplicación y es de cómo crear un grupo y añadir gastos. Para ello después de pulsar el botón de *Nuevo Tricount* nos da la opción de crear un grupo desde 0, le damos un

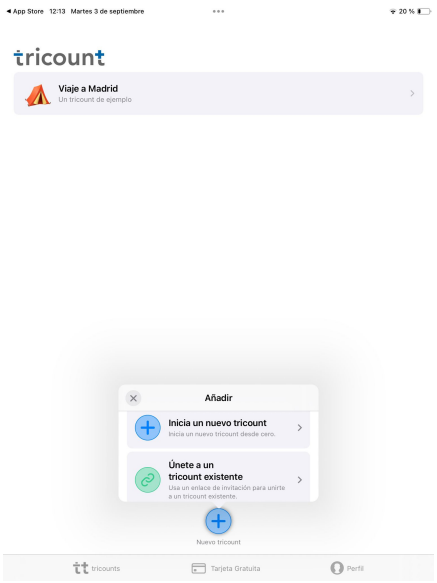


Figura 3.4: Añadir un nuevo grupo.

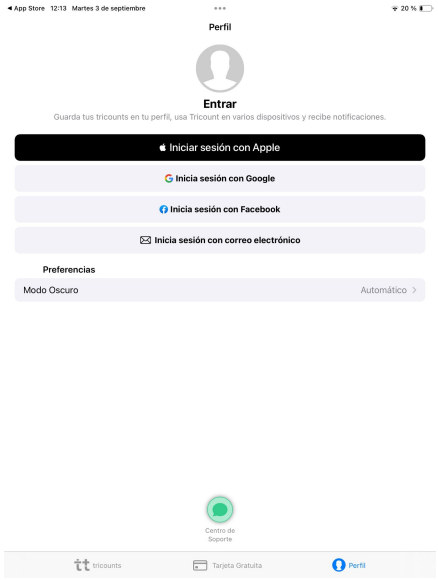


Figura 3.5: Pantalla del perfil.

nombre, añadimos los miembros que van a participar en el grupo y elegimos la divisa.(*Figura 3.6*). Una vez creado nos permite enviar un enlace a la gente que va a participar en el grupo (*Figura 3.7*).

Lo siguiente que vamos a hacer es añadir un gasto, para ello pulsamos el botón de *Añadir un gasto* y nos muestra una pantalla donde podemos añadir el nombre del gasto, la cantidad, la fecha, quien ha realizado el pago y a quien le corresponde pagar (*Figura 3.8*). Por último, nos muestra un resumen de los gastos que se han realizado y las deudas que tiene cada miembro del grupo (*Figura 3.9*).

3.2.1.3. Calificación

En la tienda oficial de Android (*Google Play*) la aplicación tiene una calificación de 4.8/5 estrellas y cuenta con más de 10 millones de descargas y en la tienda oficial de Apple (*App Store*) tiene una calificación de 4.7/5 estrellas. La mayoría de comentarios son positivos y destacan la facilidad de uso y la utilidad de la aplicación.

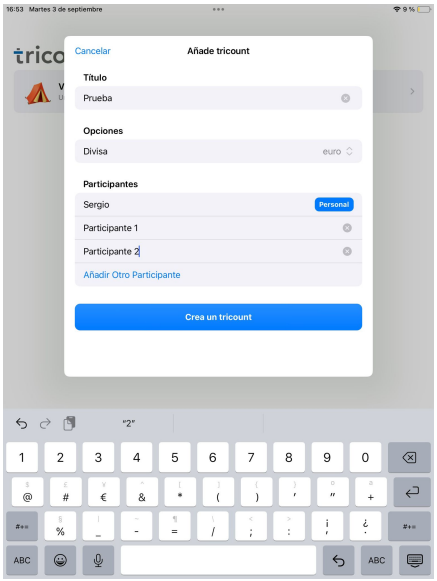


Figura 3.6: Detalles de un nuevo grupo.

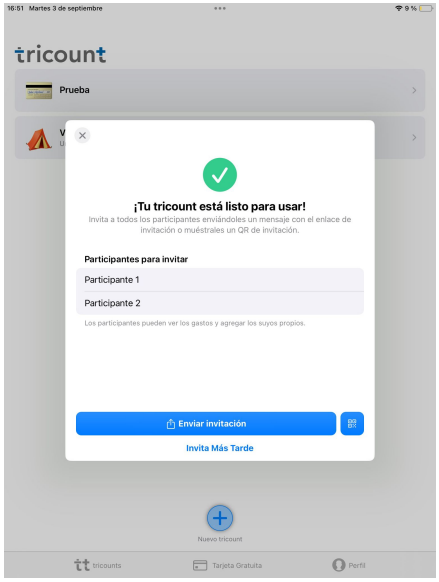


Figura 3.7: Enviar invitación a los miembros del grupo.

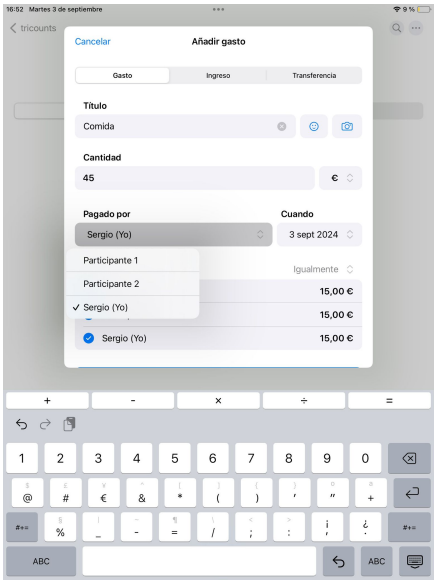


Figura 3.8: Añadir un gasto.

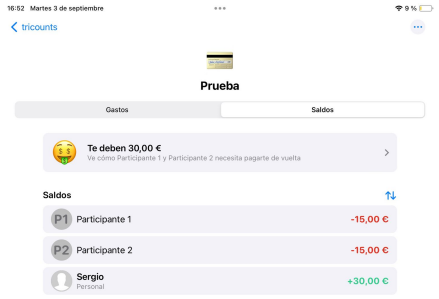


Figura 3.9: Resumen de los gastos y deudas.

3.2.2. Todoist

Todoist es una aplicación de gestión de tareas y proyectos que ayuda a los usuarios a organizar sus actividades de manera diaria. Es ampliamente utilizada tanto por personas como por equipos, debido a su flexibilidad, interfaz intuitiva y capacidad para gestionar desde simples listas de tareas hasta proyectos complejos. Está disponible para Android, iOS, Windows, Mac y la web. (Todoist, 2025)

3.2.2.1. Funciones Principales

1. **Gestión de Tareas**

    a) Añadir Tareas. Pueden crear tareas rápidamente. Cada tarea puede tener un título, una descripción y se puede asignar un proyecto o etiqueta para mantener todo organizado.

    b) Fechas de Vencimiento y Recordatorios. Se puede establecer una fechas límites para las tareas, además de recordatorios específicos, lo que ayuda a asegurarse de que las tareas se hagan a tiempo.

    c) Tareas Recurrentes. Si hay tareas que se repiten, se pueden configurar como tareas recurrentes.

2. **Proyectos y Subtareas**:

    a) Proyectos. Permite organizar las tareas dentro de proyectos. Los proyectos son carpetas donde se agrupan las tareas relacionadas con un objetivo o área específica.

    b) Subtareas. Para tareas más complejas, pueden descomponerlas en subtareas, permitiendo gestionar pasos más pequeños.

3. **Etiquetas y Filtros**

    a) Etiquetas. Permiten agrupar tareas de diferentes proyectos a través de características comunes.

    b) Filtros. Permite crear vistas personalizadas de las tareas, según criterios como fechas, etiquetas, etc.

4. **Prioridades**. Permite asignar niveles de prioridad a cada tarea, de 1 a 4

5. **Colaboración y Gestión de Equipos**

    a) Asignar Tareas. En proyectos compartidos, puedes asignar tareas a otros miembros del equipo, facilitando la colaboración en proyectos grupales.

b) Comentarios y Archivos. Los usuarios pueden añadir comentarios.

c) Compartir Proyectos. Puedes compartir proyectos con otros usuarios, lo que permite que todos los miembros vean, añadan o editen tareas dentro de ese proyecto.

6. **Integraciones**. Se integra con una amplia gama de aplicaciones y servicios, como Google Calendar, Slack, Dropbox, etc.

3.2.2.2. Modo de Uso

Lo primero que tenemos que hacer la primera que entramos en la aplicación es registrarnos a través de las diferentes opciones que nos permite la aplicación (*Figura 3.10*). Una vez completado este paso nos permite elegir el ámbito, en el que planeas usar la aplicación (he elegido personal porque es la que más se ajusta a la aplicación que estamos buscando). (*Figura 3.11*). Hecho todo esto nos lleva a la pantalla principal donde nos muestra las tareas que vencen hoy, en este caso no aparece ninguna porque aún no hemos añadido nada (*Figura 3.12*). Si le damos a añadir nos permite poner una descripción a la tarea, una fecha de vencimiento, una etiqueta, una prioridad, etc. (*Figura 3.13*). Una vez añadida nos aparece la tarea en nuestra lista de tareas pendientes donde la podemos marcar como realiza o no realizada. (*Figura 3.14*). También tenemos otra pestaña donde nos muestra un calendario con las tareas que tenemos pendientes por días, además de la tarea que hemos añadido, nos muestra tareas de ejemplo (*Figura 3.15*). Por último, cabe añadir que la aplicación cuenta con otra pestaña donde se puede buscar tareas y otra para explorar ya que podemos tener tareas en diferentes bandejas.

3.2.2.3. Calificación

En la tienda oficial de Android (*Google Play*) la aplicación tiene una calificación de 4.5/5 estrellas y cuenta con más de 10 millones de descargas y en la tienda oficial de Apple (*App Store*) tiene una calificación de 4.7/5 estrellas. La mayoría de comentarios son positivos y destacan la facilidad de uso y la utilidad de la aplicación. La versión de pago tiene un coste de 4.99€/mes o 47.99€/año.



Figura 3.10: Pantalla de registro.

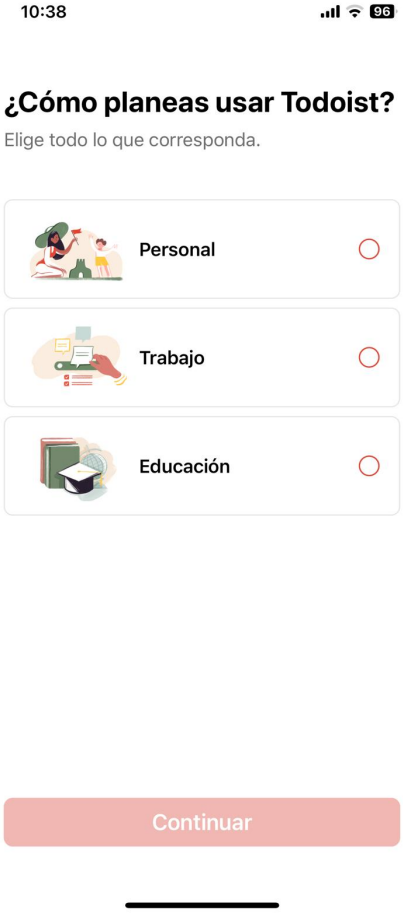


Figura 3.11: Elegir el ámbito de la aplicación.

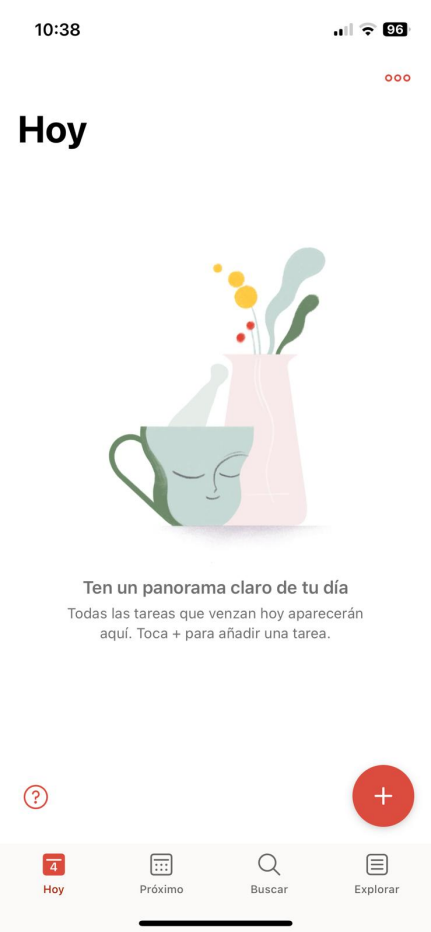


Figura 3.12: Pantalla principal.



Figura 3.13: Añadir una tarea.

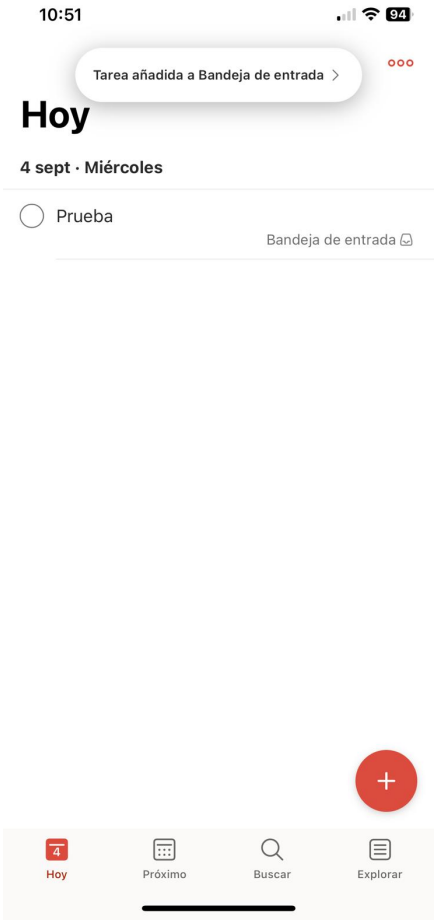


Figura 3.14: Pantalla con la tarea añadida.

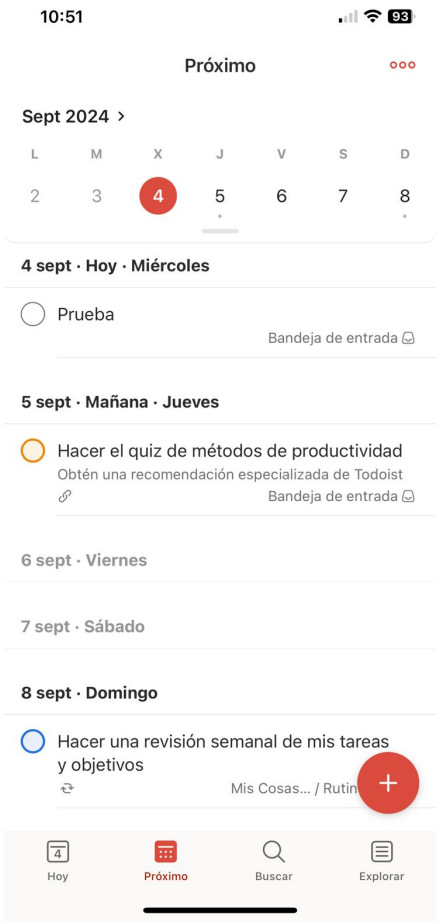


Figura 3.15: Calendario de tareas.

3.2.3. MeToca

Es una aplicación desarrollada por el gobierno de España con el objetivo de facilitar la organización de tareas del hogar de manera equitativa y eficiente. Es una iniciativa para promover la corresponsabilidad en el hogar, asegurando que las responsabilidades domésticas sean distruidas de manera justa entre todos los miembros de la familia. Es gratuita y en un principio estaba disponible para Android e iOS, pero en la actualidad solo está disponible para Android. (Ministerio de Igualdad, 2023)

3.2.3.1. Funciones Principales

1. **Gestión de Tareas del Hogar**

    a) Asignación de tareas. Permite asignar tareas a cada miembro del hogar. Las tareas pueden ser de cualquier tipo

    b) Calendario y Recordatorios. Incluye un calendario integrado donde se pueden programar las tareas frecuentes o únicas. Además envía recordatorios en caso de no hacerlas a tiempo.

2. **Seguimiento y Cumplimiento**

    a) Registro de Tareas Completadas. Cada usuario puede marcar sus tareas completadas y así poder llevar unn control del cumplimiento de responsabilidades.

    b) Estadísticas de Participación. Genera informes y estadísticas sobre cómo se están distribuyendo las tareas y si se están cumpliendo.

3. **Distribución Equitativa**

    a) Corresponsabilidad. Fomenta la corresponsabilidad en el hogar a través de la visualización y el seguimiento de tareas todos los miembros pueden ver si las responsabilidades se están distribuyendo de manera equitativa.

4. **Soporte para diferentes tipos de hogares**

    a) Adaptabilidad. Está diseñada para ser inclusiva y adapatable para cualquier tipo de hogar o de familia

3.2.3.2. Modo de Uso

En este caso no se ha podido probar la aplicación porque esta cerrada temporalmente, debido a la falta de usuarios.

3.2.3.3. Calificación

En la tienda oficial de Android (*Google Play*) la aplicación tiene una calificación de 1.6/5 estrellas, con apenas más de 10.000 descargas, aunque los creadores aseguran que tienen muchisimas funcionalidades la mayoría de comentarios aseguran que no les resulta muy útil.

3.2.4. Cozi Family Organizer

Es una aplicación diseñada para ayudar a las familias a organizar y gestionar sus actividades diarias y así organizar y gestionar sus actividades de manera colaborativa. Cozi se enfoca en la simplicidad y en proporcionar funciones esenciales para coordinar las actividades de varios miembros de una familia ocupada. Está disponible para Android, iOS y la web. Dispone de una versión gratuita que incluye la mayoría de las funcionalidades principales y una versión de pago con funciones adicionales. (Cozi Inc., 2025)

3.2.4.1. Funciones Principales

1. **Calendario Familiar**

   a) Calendario Unificado. El calendario de Cozi hace que todos los miembros de la familia tengan acceso a un calendario compartido. Cada miembro de la familia tiene un color asignado, lo que facilita ver quién tiene cada actividad y cuando tiene que hacerla.

   b) Eventos y Recordatorios. Se pueden añadir eventos y establecer recordatorios para asegurar que nadie olvide compromisos importantes, cómo citas médicas, actividades escolares o eventos familiares.

   c) Notificaciones. La aplicación envía notificaciones a los miembros de la familia sobre los eventos próximos, asegurando que todos estén informados de los planes.

2. **Lista de Compras Compartidas**

   a) Lista de Compras Colaborativas. Permite crear y compartir listas de compras entre todos los miembro de la familia, lo que permite que varias personas puedan contribuir a la lista y cualquier miembro puede marcar articulos que ya se han comprado.

   b) Actualizaciones en Tiempo Real. La lista de compras se actualiza en tiempo real, lo que significa que cualquier cambio que se haga se refleja inmediatamente en todos los dispositivos.

c) Organización por Tienda. Las listas pueden organizarse por tipo de tienda o categoría.

3. **Listas de Tareas**

   a) Tareas Pendientes. Permite crear listas para proyectos domésticos, responsabilidades diarias o cualquier tipo de actividades pendientes

   b) Marcar Completadas. Los usuarios pueden marcarla como hecha, lo que ayuda a mantener un seguimiento claro de lo que se ha logrado

4. **Planificación de Comidas**

   a) Organizador de Menús. Facilita organizar las comidas con antelación y evita el estrés diario de decidir qué cocinar.

   b) Recetas Integradas. Puedes guardar y organizar recetas directamente en la aplicación y con solo un clic.

   c) Sincronización con el calendario. Los menús planificados pueden sincronizarse con el calendario para que toda la familia lo vea.

5. **Diario Familiar**. Es una función donde los miembros pueden registrar eventos o recuerdos importantes, como fotos y notas sobre actividades familiares. Además se puede compartir con otro familiares y amigos.

3.2.4.2. Modo de Uso

Después de descargar e instalar esta aplicación, lo primero que nos muestras es una pantalla de registro/inicio de sesión (*Figura 3.16*). donde nos pide un nombre, el correo electrónico y una contraseña. Después de crear nuestra cuenta nos muestra una pantalla donde nos pide el nombre de nuestra familia y los miembros que la componen (*Figura 3.17*). Después de esto nos permite añadir un evento al calendario, con un nombre, la fecha, quién la tiene que realizar, si se repite o no y cúando queremos que nos lo recuerde (*Figura 3.18*). Una vez completado este paso nos muestra el un calendario con todas las tareas que tenemos pendientes (*Figura 3.19*). Si nos vamos a la pestaña de "List"nos muestras todas las listas de compras o de cosas que hacer ya que están divididas en dos pestañas, en la (*Figura 3.20*) podemos ver un ejemplo de lista de la compra con un árticulo añadido a una de ellas. Otra de las pestañas que nos ofrece la aplicación es la de Recetas"donde nos muestra recetas de cocina y además nos deja añadir más recetas a través de una url de la receta encontrada (*Figura 3.21*). Por último, la pestaña final es la de "Familia"donde nos permite configurar información sobre la familia, por ejemplo, añadir una foto de perfil o añadir a un miembro más a la familia.

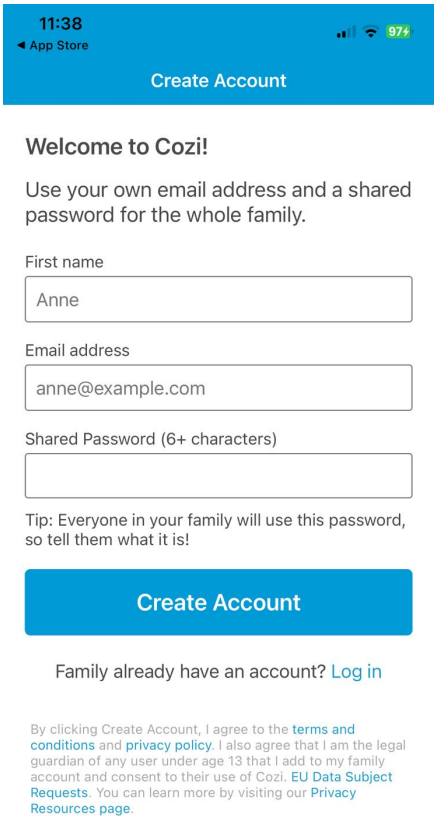


Figura 3.16: Registro.

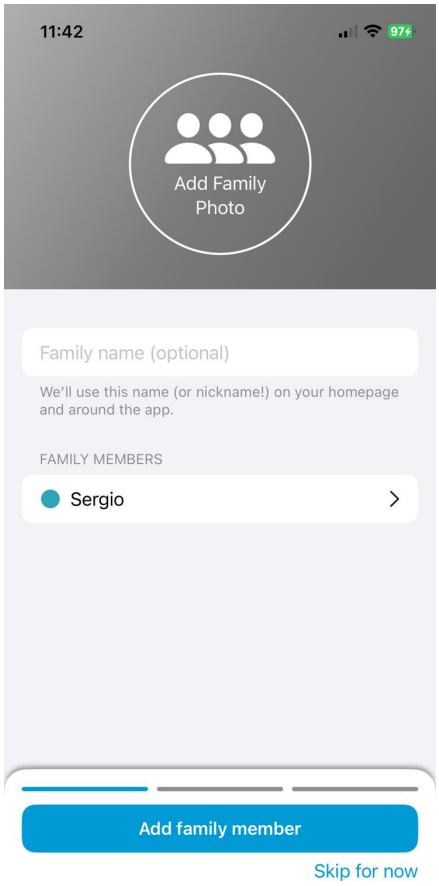


Figura 3.17: Crear una familia.

3.2.4.3. Calificación

En la tienda oficial de Android (*Google Play*) la aplicación tiene una calificación de 4.1/5 estrellas y cuenta con más de 5 millones de descargar. En la tienda oficial de Apple (*App Store*) tiene una calificación de 4.8/5 estrellas. La mayoría de comentarios son positivos y destacan la facilidad de uso y la utilidad de la aplicación. Por lo que he podido observar la página esta completamente en inglés.

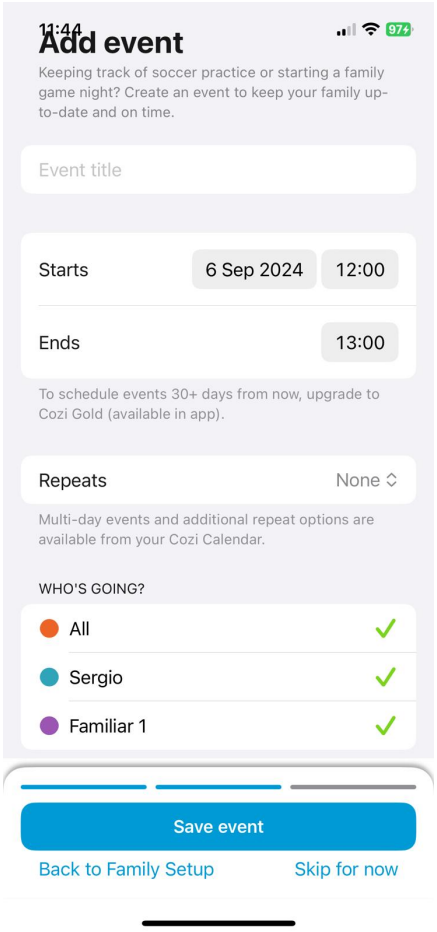


Figura 3.18: Añadir un evento.

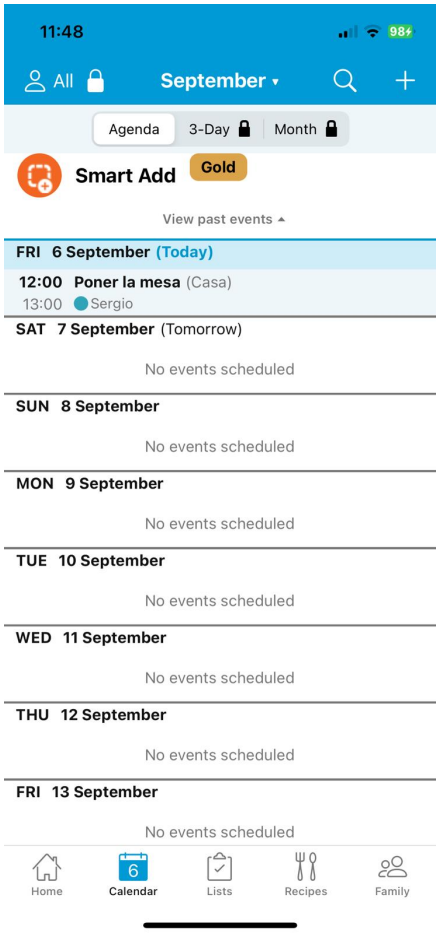


Figura 3.19: Calendario.

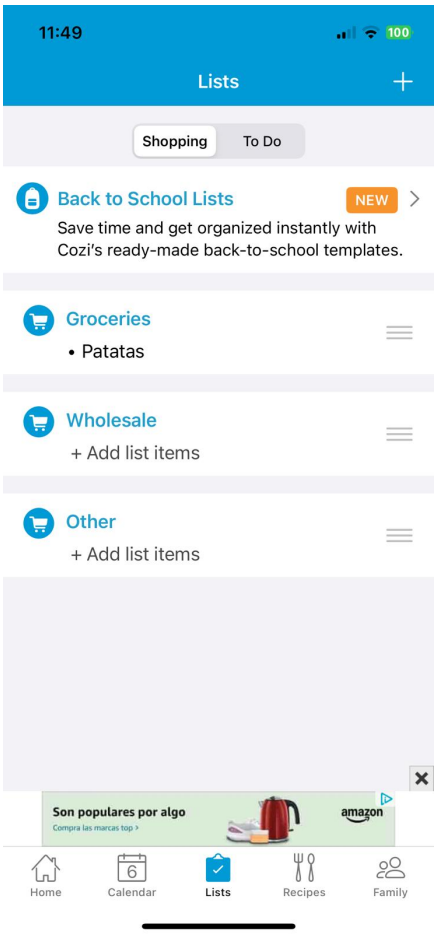


Figura 3.20: Listas de la compra.

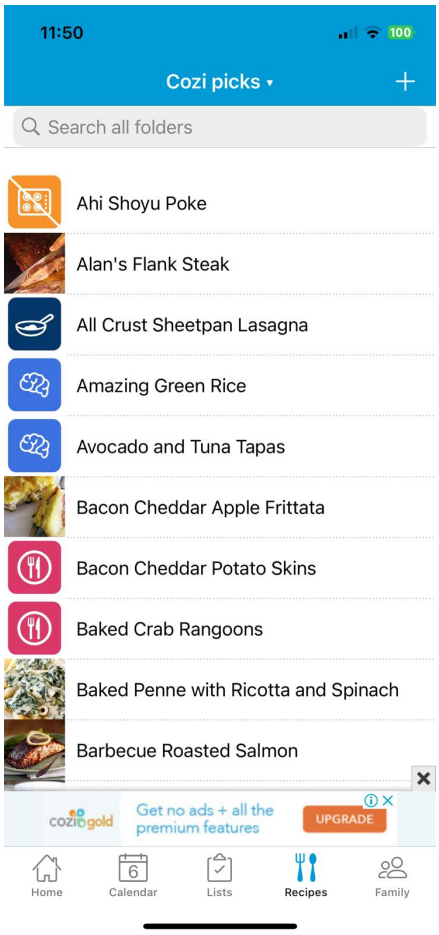


Figura 3.21: Recetas.

3.2.5. OurHome

Es una aplicación que ayuda a familias a organizar y gestionar tareas, compras y responsabilidades diarias de manera colaborativa. Está diseñada para facilitar la administración de tareas en el hogar y fomentar la cooperación familiar mediante asignación de tareas, el seguimiento de recompensas y la planificación de compras. (OurHome, 2025)

3.2.5.1. Funciones Principales

1. **Asignación de Tareas**. Permite asignar y crear tareas a los miembros de la familia, definiendo la frecuencia y quien debe hacer cada tarea y en qué plazo. Además incluye un sistema de recompensas basado en puntos, donde los miembros ganan puntos al completar tareas.
2. **Lista de Compra Compartida**. Incluye una lista de compras integrada que puede ser gestionada por cualquier miembro de la familia. Permite categorizar artículos y marcarlos como comprados.
3. **Calendario Familiar**. Incluye un calendario compartido donde se pueden añadir eventos y recordatorios, donde cada miembro puede ver y agregar eventos.
4. **Seguimiento de Objetivos**. Permite a los padres establecer objetivos específicos para los niños o cualquier miembro de la familia, cómo hábitos de estudio o rutinas diarias
5. **Sistema de notificaciones**. Los usuarios reciben notificaciones cuando se les asigna una tarea, se acerca la fecha límite o se actualiza la lista de compras.

3.2.5.2. Modo de Uso

Si nos metemos desde la página web oficial de la aplicación, nos aparece un enlace para que la descarguemos en nuestro dispositivo iOS, sin embargo, en mi dispositivo iOS no me da la opción de instalarmela, por lo tanto, tras investigar, llegué a la conclusión de que la aplicación no está disponible para nuestra región.

3.2.5.3. Calificación

En la tienda oficial de Android (*Google Play*) la aplicación tiene no tiene ningún tipo de calificación y solo cuanta con un poco más de 500 descargas. No aparece ningún tipo de comentario por lo tanto se deduce, que no es una aplicación muy popular.

3.2.6. Comparativa

A continuación se muestra una tabla comparativa de las aplicaciones analizadas, donde se resumen las principales características y funcionalidades de cada una:

Características	Tricount	Todoist	MeToca	Cozi Family Organizer	OurHome
Plataforma	Android, iOS, Web	Android, iOS, Windows, Mac, Web	Android	Android, iOS, Web	Android
Coste	Gratuito	Gratuito y Pago	Gratuito	Gratuito y Pago	Gratuito
Calificación Media	4.75	4.6	1.6	4.45	No disponible
Control de Gastos	✓	–	–	–	–
Gestión de Tareas	–	✓	✓	✓	✓
Calendario	–	✓	–	✓	✓
Lista de Compras	–	–	–	✓	✓
Asignación de Tareas	–	–	✓	✓	✓

Cuadro 3.1: Comparativa de aplicaciones para la gestión de tareas y gastos.

La Tabla 3.1 presenta una comparativa de diversas aplicaciones utilizadas para la gestión de tareas y organización en hogares compartidos. Se han seleccionado cinco aplicaciones populares que ofrecen diferentes funcionalidades para facilitar la administración de gastos, la planificación de tareas y la coordinación entre los miembros del hogar.

Cada fila de la tabla representa una característica que permite evaluar las diferencias entre las aplicaciones. En primer lugar, se muestra la **plataforma** en la que cada aplicación está disponible, lo que puede influir en la accesibilidad y compatibilidad para los usuarios. Posteriormente, se indica el **coste**, diferenciando entre aplicaciones completamente gratuitas y aquellas que ofrecen versiones premium con funcionalidades adicionales.

La columna de **calificación media** refleja la puntuación promedio dada por los usuarios en las tiendas de aplicaciones, lo que proporciona una referencia sobre la satisfacción y confiabilidad de cada herramienta. En cuanto a las funcionalidades, se incluyen características clave como el **control de gastos**, útil para administrar pagos y contribuciones entre compañeros de piso; la **gestión de tareas**, que permite asignar y organizar responsabilidades dentro del hogar; **el calendario**, que facilita la planificación de eventos o recordatorios compartidos; **la lista de compras**, ideal para coordinar la adquisición de productos necesarios; y **la asignación de tareas**, que permite distribuir responsabilidades entre los miembros del hogar.

A partir de esta comparativa, se puede observar que **Tricount** es la única aplicación que se centra exclusivamente en el control de gastos, mientras que **Todoist** destaca en la gestión de tareas, aunque no ofrece funcionalidades

específicas para hogares compartidos. Por otro lado, **Cozi Family Organizer** y **OurHome** son aplicaciones más completas, ya que integran múltiples funcionalidades como listas de compras, asignación de tareas y calendarios. Sin embargo, en términos de calificación, **MeToca** presenta una puntuación baja, lo que puede indicar posibles problemas de usabilidad o funcionalidad en comparación con el resto.

3.3. Legislación actual de protección de datos

Al desarrollar una aplicación para la gestión del hogar compartido, hay que tener en cuenta la privacidad y seguridad de los datos de los usuarios. En un entorno donde varias personas comparten responsabilidades y gastos, es inevitable manejar información como nombres, correos electrónicos, deudas pendientes e incluso hábitos de convivencia. Sin una adecuada protección de estos datos, podrían generarse situaciones de vulnerabilidad. Por ello, es fundamental que la aplicación cumpla con la Ley de Protección de Datos, garantizando que los usuarios tengan control sobre su información y que esta sea almacenada y procesada de manera segura.

La **Ley de Protección de Datos** está diseñada para garantizar el derecho a la privacidad y la protección de los datos personales, evitando el uso indebido o la divulgación de la información personal de personas. En la Unión Europea, la normativa más relevante es el **Reglamento General de Protección de Datos (GDPR)**, que entró en vigor en 2018 y establece un estándar estricto para el manejo de datos en todos los países miembros, aplicando también a empresas fuera de la UE que traten datos de ciudadanos europeos. En España, esta ley se complementa con la **Ley Orgánica de Protección de Datos y Garantía de los Derechos Digitales (LOPD-GDD)**, que adapta el GDPR pero al contexto español. (Parlamento Europeo, 2016)

Las principales medidas de este reglamento son: (Jefatura del Estado, 2018)

1. **Consentimiento explícito:** Las organizaciones deben obtener el consentimiento explícito de los usuarios antes de recopilar sus datos. Este consentimiento debe ser claro, específico e informado, lo que significa que los usuarios deben saber exactamente qué datos se recogen y para qué fines se usarán.

2. **Derecho al Acceso, Rectificación y Supresión:** Los usuarios tienen el derecho de acceder a sus datos personales, solicitar correciones o eliminarlos. Esto se conoce como *derecho al olvido* y permite a los usuarios mantener control sobre su información en línea.

3. **Transparencia y Finalidad:** Los empresas deben informar a los usuarios sobre qué datos recopilan, el propósito de dicha recopilación y durante cuánto tiempo los conservarán. Esto busca evitar el almacenamiento de excesivo de datos y el uso para fines distintos a los incialmente indicados.

4. **Seguridad de los datos:** Las organizaciones deben implementar medidas de seguridad para proteger los datos, como el cifrado y las políti-

cas de acceso restringido. Estas medidas están orientadas a minimizar el riesgo de filtraciones de datos y accesos no autorizados.

5. **Notificación de brechas de seguridad:** Si ocurre una violación de seguridad que afecte datos personales, las empresas deben notificar a las autoridades competentes y a los afectados en un plazo de 72 horas.

Estas medidas buscan proteger los derechos de privacidad de los usuarios y obligan a las empresas a adoptar prácticas de manejo de datos que respeten la seguridad, confidencialidad y transparencia.

3.4. Estudio de usabilidad y accesibilidad

3.4.1. Estudio de usabilidad

La **usabilidad** se refiere a la facilidad con la que los usuarios pueden interactuar con un producto o sistema, para alcanzar sus objetivos de manera eficiente y satisfactoria. La usabilidad se define como ün atributo de calidad que evalúa qué tan fáciles de usar son las interfaces de usuario"(Nielsen, 1993)

Las principales características de la usabilidad son (Staff, 2023):

1. **Facilidad de aprendizaje:** Capacidad del sistema para permitir que nuevos usuarios realicen tareas básicas la primera vez que lo utilizan.
2. **Eficiencia:** Una vez que los usuarios han aprendido el diseño, deben poder realizar tareas con rapidez y precisión.
3. **Memorabilidad:** Los usuarios deben poder retomar el uso del sistema después de un período de inactividad sin necesidad de reaprender todo.
4. **Manejo de errores:** El sistema debe minimizar la frecuencia y gravedad de los errores, y permitir una recuperación fácil cuando ocurran.
5. **Satisfacción:** El uso del sistema debe de ser agradable, proporcinando una experiencia positiva al usuario.

La usabilidad es fundamental para garantizar que los productos o sistemas sean accesibles y efectivos para los usuarios previstos. Un alto nivel de usabilidad conduce a una mayor satisfacción del usuario, reduce los errores y aumenta la eficiencia en la realización de tareas. Además, mejora la adopción del producto y puede proporcionar una ventaja competitiva en el mercado. Una buena usabilidad asegura que los usuarios puedan navegar e interactuar con el sistema de manera intuitiva, encontrando la información que buscan y completando acciones sin dificultades. (Adverthia, 2019)

3.4.2. Estudio de accesibilidad

La **accesibilidad** en el desarrollo de software se refiere al diseño de aplicaciones que puedan ser utilizadas por el mayor número de personas, incluyendo aquellas con discapacidades físicas, cognitivas o sensoriales. Diseñar software implica garantizar que los programas y sus funcionalidades estén disponibles para una amplia gama de usuarios, independientemente de sus capacidades. (ThoughtWorks, 2020)

Las principales características de la accesibilidad son: (Burriel, 2024)

1. **Perceptible:** La información y los componentes de la interfaz deben ser presentados de manera que los usuarios pueden percibirlos. Esto incluye proporcionar alternativas textuales para contenido no textual y asegurar que el contenido sea adaptable a diferentes formatos

2. **Operable:** Los componentes de la interfaz y la navegación deben ser utilizables por todos los usuarios. Por ejemplo, garantizar que todas las funciones sean accesibles mediante teclado y proporcionar suficiente tiempo para que los usuarios realicen tareas

3. **Comprensible:** La información y el funcionamiento de la interfazz deben ser claros y coherentes, facilitando la comprensión y el uso del sistema. Esto implica utilizar un lenguaje sencillo y predecible, así como proporcionar asistencia cuando sea necesario.

4. **Robusto:** El contenido debe ser lo suficientemente robusto como para ser interpretado de manera fiable por una amplia de agentes de usuario, incluyendo tecnologías de asistencia. Esto asegura que el contenido siga siendo accesible a medida que las tecnlogías avanzan.

Implementar la accesibilidad en el desarrollo de software garantiza que todas las personas, independientemente de sus capacidades, puedan interactuar con las aplicaciones de manera efectiva, Esto no solo cumple con normativas legales y éticas, sino que también amplía la base de usuarios potenciales y mejora la experiencia general del usuario. Además, la accesibilidad beneficia a otros grupos, como personas mayores o usuarios con limitaciones temporales, y mejora la usabilidad en diferentes contextos de uso. (Mozilla Developer Network, 2024)

Capítulo 4

Análisis de tecnologías

4.1. Análisis de las tecnologías Frontend

En la actualidad existen multitud de opciones en el mercado para el desarrollo de tecnologías móviles. A continuación se describen las tecnologías más populares y utilizadas en la actualidad para el desarrollo de aplicaciones móviles.

4.1.1. Desarrollo nativo

En primer lugar tenemos las tecnologías de **desarrollo nativo**, es decir, las tecnologías que permiten la creación de aplicaciones específicamente diseñadas para un sistema operativo en concreto. Dichas tecnologías son:

- **Kotlin.** Kotlin es un lenguaje que permite desarrollar aplicaciones para todas las plataforams soportadas por Android, siendo Android Studio el entorno de desarrollo integrado (IDE) principal para la creación de aplicaciones nativas en dicho sistema operativo. Una de las ventajas más destacas es su total interoperabilidad con Java, lo que facilita el uso de bibliotecas escritas en Java dentro de proyectos desarrollados en Kotlin. Además, el lenguaje proporciona mecanismos avanzados de seguridad frente a valores nulos, lo que ayuda a prevenir errores durante la ejecución de las aplicaciones. Entre sus deventajas destaca que, aunque la popularidad ha crecido, aún no alcanzar el nivel de adopción de Java, lo que puede dificultar encontrar soluciones a problemas específicos. Asimismo, aunque existen iniciativas para su uso multiplataforma, Kotlin sigue estando principal,emte orientado al desarrollo de aplicaciones en Android. (JetBrains, 2025)
- **Swift.** Es un lenguaje, diseñado específicamente para crear aplicacio-



Figura 4.1: Logo de Kotlin

nes de alto rendimiento y con una sintaxis clara y moderna. En este caso el IDE es XCode utilizado para el desarrollo nativo en las plataformas de Aple, como iOS y macOS. Swift se caracteriza por su seguridad, facilidad de uso y capacidades de optimización, lo que lo convierte en una buena opción para el desarrolo de aplicaciones en el ecosistema de Apple. Como principales carácteristicas de Swift podemos destacar una **sintaxis simple y moderna**, un **alto rendimiento** debido a su compilador avanzado permite ejecutar aplicaciones de manera eficiente, **seguridad** gracias a su sistema de tipos fuertes y a la prevención de errores en tiempo de ejecución, e **interoperabilidad con Objetive-C** lo que facilita la integración de código. Sin embargo, una de las principales deventajas de Swift es similar a la mencionada anteriormente en Kotlin, **la duplicidad de esfuerzos** en el desarrollo de aplicaciones multiplataforma. Las aplicaciones desarrolladas con Swift obligan a los desarrolladors a crear aplicaciones independientes para otras plataformas. (Apple Inc., 2025)



Figura 4.2: Logo de Swift

4.1.2. Desarrollo Híbrido

Las tecnologías de desarrollo híbrido representan una alternativa eficaz en el ámbito de aplicaciones multiplataforma. Estas tecnologías permiten crear una única base de código que puede ser ejecutada tanto en dispositivos con sistema operativo iOS como en Android. La principal ventaja de este enfoque es la optimización de los recursos, tanto económicos como operativos, ya que reduce significativamente los costes asociados al desarrollo y mantenimiento de las aplicaciones. Al utilizar un solo lenguaje y una única estructura de programación, se consigue implementar una solución funcional para ambas plataformas. Además, permite una mejor experiencia al usuario, garantizando que la aplicación tenga un comportamiento y apariencia similares. Algunos ejemplos de esta tecnología son:

- **Ionic.** Es un framework de código abierto que permite crear aplica-

ciones móviles, web y de escritorio utilizando tecnologías webs, como HTML, CSS y JavaScript. Se integra con diferentes frameworks frontend como Angular React o Vue, lo que permite a los desarrolladores utilizar el conjunto de herramientas y librerías de su preferencia. Además, Ionic tiene componentes nativos que proporcionan una biblioteca de elementos visuales optimizados para dispositivos móviles. Cuenta también con una **amplia comunidad** y herramientas adicionales para la integración con servicios de backend y otras tecnologías. (Ionic Framework, 2025)



Figura 4.3: Logo de Ionic

- **Flutter.** Flutter es un framework de código abierto desarrollado por Google que permite crear aplicaciones nativas multiplatadorma con una única base de código. Utiliza el lenguaje de programación Darrt y proporciona un enfoque único para desarrollar aplicaciones móviles, web y de escritorio. Tiene un **alto rendimiento** debido a que utiliza un motor gráfico y compila directamente a código nativo, lo que garantizando un rendimiento similar al de aplicaciones nativas. Además , cuenta con *Hot Reload* que facilita el desarrollo rápido al permitir que lo cambios en el código se reflejen instantáneamente en la aplicación sin tener que recompilar por completo. (Google, 2025b)



Figura 4.4: Logo de Flutter

- **React Native.** React Native es un framework de código abierto desarrollado por Facebook que permite crear aplicaciones móviles nativas utilizando JavaScript y React. A diferencia de otros frameworks híbridos, React Native no se limita a emular una experiencia nativa, en su lugar, **genera componentes nativos**, lo que permite un rendimiento más cercano al de una aplicación pura. Además cuenta con una **amplia comunidad** y una gran cantidad de librerías y herramientas que facilitan el desarrollo de aplicaciones. (Meta Platforms, 2025)

- **Xamarin.** Es un framework de código abierto desarrollado por Microsoft que permite crear aplicaciones móviles nativas para iOS y Android



Figura 4.5: Logo de React Native

utilizando C# y .NET. Xamarin permite compartir hasta un 90 % del código. Además, proporciona acceso a las API nativas de cada plataforma, lo que permite a los desarrolladores crear aplicaciones con una **experiencia de usuario nativa**. También se integra a la perfección con herramientas del ecosistema de Microsoft, como Visual Studio, facilitando el desarrollo y la depuración de aplicaciones. (Microsoft, 2025)



Figura 4.6: Logo de Xamarin

4.1.3. Tabla comparativa de tecnologías front-end

La siguiente tabla ofrece una comparación entre las tecnologías más populares en el desarrollo frontend de aplicaciones móviles, evaluando aspectos clave como el lenguaje de programación, rendimiento, disponibilidad de componentes de UI, tamaño de la comunidad, soporte de funcionalidad *Hot Reload* e integración con otros sistemas y herramientas. Esta información proporciona una visión global de las fortalezas y limitaciones de cada opción, facilitando así la selección de la tecnología que mejor se ajuste a los requisitos específicos del proyecto.

Tecnología	Lenguaje	Rendimiento	Componentes UI	Comunidad	Hot Reload	Integración
Kotlin	Java	Nativo	Componentes Nativos	Amplia	No	Herramientas Nativas
Swift	Swift	Nativo	Componentes Nativos	Amplia	No	Herramientas Nativas
Ionic	JavaScript	Bueno	Basado en Web	Amplia	Sí	Buen Soporte
Flutter	Dart	Cercano a Nativo	Personalizados	Activa	Sí	Buen Soporte
React Native	JavaScript	Bueno	Componentes Nativos	Amplia	Sí	Buen Soporte
Xamarin	C#	Bueno	Componentes Nativos	Amplia	Sí	Integración Microsoft

Cuadro 4.1: Comparativa de tecnologías para desarrollo de aplicaciones móviles.

En conclusión, la tabla destaca cómo cada tecnología ofrece un conjunto único de beneficios. Las opciones nativas como Kotlin y Swift son preferibles para aplicaciones que requieran un rendimiento óptimo y un uso extensivo de componentes específicos de la plataforma. Sin embargo, soluciones como Flutter y React Native se presentan como alternativas atractivas para el desarrollo multiplataforma debido a su rendimiento cercano al nativo, su comunidad activa y sus herramientas avanzadas de desarrollo rápido como *Hot Reload*.

4.2. Analisis de las tecnologías Backend

Otra de las decisiones que hay que tomar es la tecnología back-end que permita la comunicación entre la aplicación y la base de datos. A continuación vamos a ver las diferentes tecnologías que hay para ello:

4.2.1. Firebase

Firebase es una plataforma de Google que ofrece una amplia variedad de servicios backend, lo que incluye **bases de datos en tiempo real** como FireStore y Realtime Database. Además, proporciona servicios de autenticación, almacenamiento de archivos, notificaciones push, análisis y etc. No requiere gestión de servidores debido a que es completamente serverless. Cuenta con una **fácil integración** con Flutter debido a que tiene SDks nativos para dicha tecnología, lo que simplifica mucho el desarrollo en comparación con otras tecnologías que requieren la integración manual. Sin embargo, Firebase tiene un **coste** asociado al uso de sus servicios, lo que puede ser un inconveniente para proyectos con presupuestos limitados. También tiene una **dependencia de Google**, porque todo esta alojados en la infraestructura de Google, lo que puede generar dependecia a largo plazo. (Google, 2025a)



Figura 4.7: Logo de Firebase

4.2.2. Node.js con Express

Node.js es un entorno de ejecución de JavaScript que permite ejecutar código JavaScript en el backend. Express es un framework ligero que permite construir aplicaciones web y APIs REST de manera rápida y sencilla. Además, es bastante útil para aplicaciones que manejan un alto número de solicitudes simultáneas. Para el almacenamiento de datos se pueden usar bases de datos realacionales como **MySQL** o **PostgreSQL** o bases de datos NoSQL como **MongoDB**. Gracias a npm (*Node Package Manager*), hay miles de paquetes disponibles que permiten acelerar el proceso de desarrollo. También es una tecnología **flexible**, porque permite elegir el tipo de base de datos. Por el contrario tiene una **Complejidad de gestión del servidor**. Aunque Express es minimalista, la creación y gestión de servidores, puede

ser más compleja en comparación con soluciones serverless. Además, la **curva de aprendizaje** puede ser más pronunciada para desarrolladores que no tienen experiencia previa con JavaScript. (Node.js Foundation, 2025)



Figura 4.8: Logo de Node.js

4.2.3. Django

Es un framework web de alto nivel escrito en Python que facilita la creación de aplicaciones webs seguras y escalables. Junto con **Django REST Framework**, permite exponer APIs REST de forma rápida y eficiente. Se integra bien con bases de datos relacionales como **PostgreSQL** y **MySQL**. Django proporciona un **rápido desarrollo** debido a que incluye una gran cantidad de herramientas listas para usar (*autenticación, administración, etc.*). Además, tiene carácteristicas de seguridad incorporadas, como protección CSRF, gestión de contraseñas seguras y más. También proporciona **gestión automática de bases de datos** sin tener que escribir consultas SQL Manuales. Sin embargo, Django puede ser algo pesado para proyectos más pequeños o simples. Además, la **curva de aprendizaje** puede ser pronunciada para desarrolladores que no tienen experiencia previa con Python. (Foundation, 2025)



Figura 4.9: Logo de Django

4.2.4. Ruby on Rails

Ruby on Rails es un framework MVC (*Modelo-Vista-Controlador*) escrito en Ruby, que permite desarrollar aplicaciones web y APis de manera rápida. Rails se integra perfectamente con bases de datos relacionales como **PostgreSQL** o **MySQL**. Una facilidad que incorpora Ruby on Rails es la **Convención sobre configuración**, lo que significa que muchas decisiones están predefinidas para que el desarrollador no tenga que configurarlas

manualmente, acelerando el proceso de desarrollo. También **facilita la interacción** con la base de datos relacionales. Sin embargo, Aunque Rails es excelente para aplicaciones web, no es tan común en el desarrollo de APIs para aplicaciones móviles. (Ruby on Rails, 2025)



Figura 4.10: Logo de Ruby on Rails

4.2.5. Tabla comparativa de tecnologías back-end

La tabla siguiente compara algunas de las tecnologías back-end más comunes en el desarrollo de aplicaciones web y móviles, evaluando aspectos como el lenguaje de programación, las bases de datos soportadas, la gestión del estado, la integración con APIs y la popularidad general.

Tecnología	Lenguaje	Base de Datos Soportada	Gestión del Estado	Integración con API	Popularidad
Firebase	-	Firebase, Realtime DB	Serverless, en tiempo real	APIs nativas de Firebase	Muy alta
Node.js	JavaScript	MongoDB, MySQL, PostgreSQL	Depende de la implementación	Muy flexible	Muy alta
Django	Python	PostgreSQL, MySQL	ORM Propio	Django REST Framework	Alta
Ruby on Rails	Ruby	PostgreSQL, MySQL	ActiveRecord ORM	RESTful API fácil de implementar	Media

Cuadro 4.2: Comparativa de tecnologías para el desarrollo back-end.

En conclusión, esta tabla muestra cómo cada tecnología back-end aporta ventajas específicas. Firebase destaca en aplicaciones en tiempo real y serverless, ideal para proyectos rápidos que requieren menos mantenimiento de servidores. Node.js es muy flexible y compatible con diferentes bases de datos, adaptándose bien a aplicaciones escalables. Django y Ruby on Rails ofrecen una estructura robusta y sencilla de implementar, siendo opciones populares para aplicaciones que necesitan un backend consolidado.

4.3. Justificación de la elección de tecnologías

Tras analizar las diferentes tecnologías disponibles para el desarrollo de la aplicación, he decidido utilizar **Flutter** como tecnología front-end porque presenta varias ventajas significativas frente al resto de alternativas. Una de las razones más destacadas es su capacidad de desarrollo multiplataforma, que permite crear una única base de código que se ejecuta tanto en Android, como en iOS, web y aplicaciones de escritorio, lo que ahorra tiempo y esfuerzo en comparación con desarrollar código nativo para cada plataforma por separado. Además, Flutter ofrece un rendimiento cercano al de las aplicaciones nativas, gracias a su motor gráfico Dart, que se compila en código nativo. Otra ventaja es el sistema de Hot Reload, que permite a los desarrolladores visualizar los cambios en tiempo real, sin tener que recompilar por completo. Tiene también una enorme capacidad de personalización, simplificando la tarea. Además, al estar respaldado por Google,, recibe soporte continuo.

En cuanto a la tecnología back-end, he decidido utilizar **Firebase** porque ofrece una serie de ventajas que lo hacen destacar frente a las otras alternativas. Para empezar, Firebase proporciona un backend completo, lo que significa que no es necesario gestionar servidores ni preocuparse por la configuración y el mantenimiento. La integración de la autenticación de usuarios es también uno de sus puntos fuertes, ya que Firebase Auth facilita la implementación de los mismos. Otra razón para elegir Firebase es su base de datos en tiempo real, que permite sicronización instantánea de datos. Además, tiene una gran integración con servicios de Google, optimizando la monitorización del rendimiento.

Capítulo 5

Análisis de metodologías de desarrollo

Las metodologías de desarrollo software han evoluciado con el tiempo para adaptarse a las diferentes necesidades y contextos de proyectos. En función de las características del proyecto y del equipo de desarrollo, será más conveniente usar una u otra.

En este capítulo se analizarán las metodologías de desarrollo de software más utilizadas en la actualidad. Se describirán las características de cada una de ellas, así como sus ventajas y desventajas. Además, se realizará una comparativa entre las metodologías más populares para determinar cuál es la más adecuada para el desarrollo de la aplicación.

5.1. Metodologías de desarrollo de software

Las metodologías de desarrollo de software son un conjunto de prácticas y técnicas que se utilizan para planificar, diseñar, implementar y probar un proyecto de software. A continuación, se describen las metodologías más utilizadas en la actualidad:

5.1.1. Modelo en cascada (Waterfall)

Una de las metodologías más tradicionales y conocidas es el **Modelo en Cascada** (Waterfall), el cual sigue un enfoque secuencial en el que cada fase del desarrollo debe completarse antes de avanzar a la siguiente. Esta metodología se compone principalmente de las etapas de recopilación de requisitos, diseño, implementación, pruebas, despliegue y mantenimiento. Su principal ventaja es la claridad de los requisitos desde el inicio, lo que facilita una bue-

na planificación y documentación. Sin embargo, su principal inconveniente es la falta de flexibilidad, ya que cualquier cambio en los requisitos implica volver a fases anteriores, lo que puede generar retrasos y costos adicionales. Esta metodología sigue siedno utilizada en proyectos donde los requisitos están bien definidos y no se esperan cambios significativos. (Sommerville, 2015)

5.1.2. Modelo en espiral

El **Modelo en Espiral** introduce un enfoque iterativo que prioriza la gestión de riegos. En cada iteración se realiza actividades de planificación, análisis de riegos, implementación y evaluación, lo que permite detectar y mitigar problemas antes de que se conviertan en fallos grandes. Esta metodología es útil en proyectos grandes y complejos, donde la incertidumbre es alta. No obstante su aplicación suele ser costosa y requiere un gestión constante del proceso, lo que lo hace menos adecuado para proyectos pequeños. (Boehm, 1988)

5.1.3. Metodología iterativa

La **metodología iterativa** se basa en el desarrollo por ciclos repetitivos, donde en cada iteración se revisa, prueba y ajusta el software hasta alcanzar el resultado deseado. A diferencia del modelo en cascada, no requiere que todas las fases de desarrollo se completen antes de volver a una anterior, lo que permite una mayor flexibilidad y adaptabilidad. Esta metodología es adecuada para proyectos donde los requisitos no están bien definidos o pueden cambiar a lo largo del tiempo. (Atlassian, 2025)

5.1.4. Desarrollo iterativo e Incremental

El **Desarrollo Iterativo e Incremental** es otra alternativa que busca dividir el desarrollo en incrementos funcionales, donde cada iteración añade nuevas carácteristicas al sistema. A diferencia del modelo en cascada, este enfoque permite obtener retroalimentación continua y realizar ajustes en cada fase del proyecto. Su flexibilidad es una de sus mejores ventajas, ya que se pueden introducir mejoras en el software en función de las necesidades del usuario. Sin embargo, gestionar adecuadamente un proyecto basado en interaciones puede ser complejo y es díficil predecir costos y tiempos de finalización con precisión. (Larman, 2003)

5.1.5. Metodología Ágil

Recientemente, las **metodologías ágiles** han ganado popularidad en la industria del software debido a su capacidad de adaptación y entrega continua de software funcional. Estas metodologías incluyen enfoques como *Scrum*, que se basa en ciclos cortos de desarrollo llamadas *sprints*, y *Extreme Programming* (XP), que se centra en la calidad del código y la colaboración entre los miembros del equipo. También destaca *Kanban*, que utiliza tableros visuales para gestionar el flujo de trabajo. (Beck, 1999)

5.1.6. DevOps

Por último, el enfoque **DevOps** ha emergido como una metodología clave en la industria del software, ya que integra el desarrollo y las operaciones en un único flujo de trabajo. Su principal objetivo es mejorar la eficiencia del desarrollo y la entrega de software, así como la colaboración entre los equipos de desarrollo y operaciones. Sin embargo, su adopción puede ser un reto para organizadores que no cuentan con la infraestructura y cultura adecuada. (Kim, Humble, Debois, y Willis, 2016)

5.2. Comparativa de metodologías

A continuación, se presenta una comparativa entre las metodologías de desarrollo de software más utilizadas en la actualidad:

Metodología	Flexibilidad	Gestión de Riesgos	Coste	Tiempo	Adaptabilidad	Documentación
Waterfall	Baja	Baja	Bajo	Largo	Baja	Alta
Modelo en Espiral	Alta	Muy Alta	Alto	Largo	Alta	Media
Iterativo	Alta	Media	Medio	Medio	Alta	Media
Iterativo e Incremental	Alta	Media	Medio	Medio	Alta	Media
Ágil	Muy Alta	Alta	Variable	Corto	Muy Alta	Baja
DevOps	Alta	Alta	Variable	Variable	Muy Alta	Baja

Cuadro 5.1: Comparativa de metodologías de desarrollo de software.

Los aparatados de la tabla 5.1 se describen a continuación:

1. **Flexibilidad:** Indica la capacidad de la metodología para adaptarse a cambios durante el proceso de desarrollo. Por ejemplo, las metodologías Ágil y DevOps tienen una flexibilidad muy alta, lo que significa que pueden adaptarse rápidamente a los cambios.

2. **Gestión de Riesgos:** Se refiere a la capacidad de la metodología para identificar, evaluar y mitigar riesgos durante el desarrollo del proyecto. El Modelo en Espiral tiene una gestión de riesgos muy alta, lo que significa que se enfoca mucho en la identificación y mitigación de riesgos.

3. **Coste:** Representa el costo asociado con la implementación de la metodología. Puede variar desde bajo hasta alto. Por ejemplo, el Modelo en Espiral tiene un coste alto debido a su enfoque en la gestión de riesgos y la iteración.

4. **Tiempo:** Indica la duración del proceso de desarrollo. Puede variar desde corto hasta largo. Por ejemplo, el Modelo en Cascada tiene un tiempo largo debido a su enfoque secuencial.

5. **Adaptabilidad:** Se refiere a la capacidad de la metodología para adaptarse a diferentes tipos de proyectos y entornos. Las metodologías Ágil y DevOps tienen una adaptabilidad muy alta, lo que significa que pueden ser aplicadas en una amplia variedad de contextos.

6. **Documentación:** Indica el nivel de documentación requerido por la metodología. Por ejemplo, la metodología Waterfall requiere una alta cantidad de documentación, mientras que Ágil y DevOps requieren menos documentación.

En conclusión, la elección de una metodología de desarrollo de software dependerá de las características del proyecto, el equipo de desarrollo y las necesidades del cliente.

5.3. Justificación de la metodología

Para mi proyecto he decidido usar la metodología iterativa, ya que permite una mayor flexibilidad y adaptabilidad a lo largo del desarrollo. Dado que los requisitos del proyecto pueden cambiar y evolucionar con el tiempo, esta metodología me permitirá revisar, probar y ajustar el software en cada iteración hasta alcanzar el resultado deseado. Además, la posibilidad de realizar ajustes continuos y obtener retroalimentación en cada ciclo de desarrollo facilitará la mejora constante del proyecto.

Capítulo 6

Análisis de la aplicación

6.1. Especificación de requisitos

La especificación de requisitos es una etapa clave en el desarrollo de un proyecto, ya que en ella se definen las funcionalidades y condiciones que debe cumplir el sistema. Su objetivo principal es establecer de manera clara y estructurada las expectativas del producto final.

En esta sección se identifican y detallan los requisitos, tomando en cuenta las necesidades y restricciones detalladas por los usuarios y demás partes interesadas. Para ello, es fundamental realizar un análisis que permita comprender qué características son esenciales para el correcto funcionamiento del software.

Cada requisito contará con un código único que permitirá clasificarlo y referenciarlo fácilmente a lo largo del documento. Este método de organización facilita su seguimiento y garantiza que todas las especificaciones sean consideradas durante el desarrollo del proyecto.

6.1.1. Requisitos funcionales

Los requisitos funcionales son especificaciones detalladas que describen las funciones y comportamientos que un sistema debe cumplir para satisfacer las necesidades del usuario. Estos requisitos definen las acciones que el sistema debe ser capaz de realizar, incluyendo las interacciones con el usuario y las respuestas a entradas específicas. Estos requisitos son ensenciales para guiar el desarrollo del sistema y asegurar que cumpla con las expectivas del usuario. (PMO Informática, 2017)

Vamos a clasificar estos requisitos en diferentes grupos según su funcionalidad.

- **Grupo 1:** Gestión de sesión. En este grupo se incluyen los requisitos relacionados con la autenticación y la gestión de la sesión del usuario.
- **Grupo 2:** Gestión del hogar. En este grupo se incluyen los requisitos relacionados con la creación y gestión de un hogar compartido.
- **Grupo 3:** Gestión de tareas. En este grupo se incluyen los requisitos relacionados con la creación y asignación de tareas en el hogar.
- **Grupo 4:** Gestión de gastos. En este grupo se incluyen los requisitos relacionados con la creación y gestión de gastos en el hogar.
- **Grupo 5:** Gestión de compra. En este grupo se incluyen los requisitos relacionados con la creación y gestión de listas de compra.

A continuación, se detallan los requisitos funcionales de cada grupo.

6.1.1.1. Grupo 1: Gestión de sesión

RF_01	**Registro**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario registrarse.
Criterios de aceptación	1. La interfaz de registro debe contener los campos: nombre, apellidos, correo electrónico y contraseña. 2. Verificar que los campos no estén vacíos. 3. Verificar que la información introducida sea válida y que los campos cumplan con el formato requerido. 4. Si los campos no son válidos o ya pertenecen a un usuario registrado, mostrar un mensaje de error. 5. Tras el registro, se mostrará un mensaje indicando que debe verificar su correo electrónico y se redigirá a la pantalla de inicio de sesión.

Cuadro 6.1: RF_01

RF_02	**Inicio de sesión**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario iniciar sesión.
Criterios de aceptación	1. La interfaz de inicio de sesión debe contener los campos: correo electrónico y contraseña. 2. Verificar que los campos no estén vacíos. 3. Verificar que la información introducida sea válida y que los campos cumplan con el formato requerido. 4. Si los campos no son válidos o no pertenecen a un usuario registrado, mostrar un mensaje de error. 5. Si el usuario no ha verificado su correo electrónico, mostrar un mensaje indicando que debe verificarlo. 6. Tras el inicio de sesión, se redirigirá a la pantalla principal de la aplicación.

Cuadro 6.2: RF_02

RF_03	**Inicio de sesión con Google**
Prioridad	Baja
Descripción	El sistema debe permitir al usuario iniciar sesión con Google
Criterios de aceptación	1. La interfaz de inicio de sesión debe contener un botón para iniciar sesión con Google. 2. Al pulsar el botón, se abrirá una ventana emergente para iniciar sesión con la cuenta de Google. 3. Si el usuario ya ha iniciado sesión con Google, se redirigirá a la pantalla principal de la aplicación. 4. Si el usuario no ha iniciado sesión con Google, se registrará automáticamente y se redirigirá a la pantalla principal de la aplicación.

Cuadro 6.3: RF_03

RF_04	**Cerrar Sesión**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario cerrar sesión
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para cerrar sesión. 2. Al pulsar el botón, se mostrará un mensaje de confirmación. 3. Si el usuario confirma, se cerrará la sesión y se redirigirá a la pantalla de inicio de sesión.

Cuadro 6.4: RF_04

RF_05	**Ver datos de usuario**
Prioridad	Media
Descripción	El sistema debe permitir al usuario ver sus datos personales.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una sección para ver los datos personales del usuario. 2. Al acceder a la sección, se mostrarán los campos: nombre, apellidos, correo electrónico y cualquier otra información relevante. 3. Si no hay datos disponibles, mostrar un mensaje indicando que no hay datos disponibles.

Cuadro 6.5: RF_05

RF_06	**Editar datos de usuario**
Prioridad	Media
Descripción	El sistema debe permitir al usuario editar sus datos personales.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para editar los datos personales del usuario. 2. Al pulsar el botón, se mostrará un formulario con los campos: nombre, apellidos, correo electrónico y cualquier otra información relevante. 3. Verificar que los campos no estén vacíos. 4. Si los campos no son válidos, mostrar un mensaje de error. 5. Tras la edición de los datos, se actualizará la información en la base de datos y se redirigirá a la pantalla principal.

Cuadro 6.6: RF_06

6.1.1.2. Grupo 2: Gestión del hogar

RF_07	**Crear hogar**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario crear un hogar compartido.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para crear un hogar. 2. Al pulsar el botón, se mostrará un formulario con los campos: nombre del hogar y nombre de los miembros que lo componen. 3. Verificar que los campos no estén vacíos. 4. Si los campos no son válidos, mostrar un mensaje de error. 5. Tras la creación del hogar, se redirigirá a la pantalla principal del hogar.

Cuadro 6.7: RF_07

RF_08	**Unirse a un hogar**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario unirse a un hogar compartido.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para unirse a un hogar. 2. Al pulsar el botón, se mostrará un formulario con el campo: código del hogar. 3. Verificar que el campo no esté vacío. 4. Si el campo no es válido, mostrar un mensaje de error. 5. Tras unirse al hogar, se redirigirá a la pantalla principal del hogar.

Cuadro 6.8: RF_08

RF_09	**Abandonar hogar**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario abandonar un hogar compartido.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para abandonar el hogar. 2. Al pulsar el botón, se mostrará un mensaje de confirmación. 3. Si el usuario confirma, abandonará el hogar y se redirigirá a la pantalla principal de la aplicación.

Cuadro 6.9: RF_09

RF_10	**Ver datos del hogar**
Prioridad	Media
Descripción	El sistema debe permitir al usuario ver los datos del hogar.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una sección para ver los datos del hogar. 2. Al acceder a la sección, se mostrarán los campos: nombre del hogar, miembros del hogar y cualquier otra información relevante. 3. Si no hay datos disponibles, mostrar un mensaje indicando que no hay datos disponibles.

Cuadro 6.10: RF_10

6.1.1.3. Grupo 3: Gestión de tareas

RF_11	**Crear tarea semanal**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario crear una tarea semanal.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para crear una tarea. 2. Al pulsar el botón, nos dará a elegir que tipo de tarea queremos crear, si semanal o puntual. 3. Al elegir la opción semanal, se mostrará un formulario con los campos: nombre de la tarea, descripción, semanas en las que se realizará y los miembros que la realizaran cada semana. 4. Verificar que los campos no estén vacíos. 5. Si los campos no son válidos, mostrar un mensaje de error. 6. Tras la creación de la tarea, se redirigirá a la pantalla principal del hogar.

Cuadro 6.11: RF_11

RF_12	**Crear tarea puntual**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario crear una tarea puntual.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para crear una tarea. 2. Al pulsar el botón, nos dará a elegir que tipo de tarea queremos crear, si semanal o puntual. 3. Al elegir la opción puntual, se mostrará un formulario con los campos: nombre de la tarea, descripción, fecha y los miembros que la realizaran. 4. Verificar que los campos no estén vacíos. 5. Si los campos no son válidos, mostrar un mensaje de error. 6. Tras la creación de la tarea, se redirigirá a la pantalla principal del hogar.

Cuadro 6.12: RF_12

RF_13	**Consultar planificación semanal**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario poder consultar las tareas asignadas por semanas.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un calendario con las tareas asignadas por semanas. 2. Al pulsar en una tarea, se mostrará la información detallada de la misma. 3. Si no hay tareas asignadas en una semana, mostrar un mensaje indicando que no hay tareas asignadas.

Cuadro 6.13: RF_13

RF_14	**Marcar tarea como realizada**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario poder marcar una tarea como realizada.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para marcar una tarea como realizada. 2. Al pulsar el botón, se mostrará un mensaje de confirmación. 3. Si el usuario confirma, la tarea se marcará como realizada y se actualizará la información en la base de datos. 4. Si el usuario cancela, no se realizará ninguna acción.

Cuadro 6.14: RF_14

RF_15	Modificar Tarea
Prioridad	Media
Descripción	El sistema debe permitir al usuario poder modificar una tarea.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para modificar una tarea. 2. Al pulsar el botón, se mostrará un formulario con los campos de la tarea a modificar. 3. Verificar que los campos no estén vacíos. 4. Si los campos no son válidos, mostrar un mensaje de error. 5. Tras la modificación de la tarea, se redirigirá a la pantalla principal del hogar.

Cuadro 6.15: RF_15

RF_16	Eliminar Tarea
Prioridad	Alta
Descripción	El sistema debe permitir al usuario poder eliminar una tarea.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para eliminar una tarea. 2. Al pulsar el botón, se mostrará un mensaje de confirmación. 3. Si el usuario confirma, la tarea se eliminará y se actualizará la información en la base de datos. 4. Si el usuario cancela, no se realizará ninguna acción.

Cuadro 6.16: RF_16

RF_17	**Ver datos de una tarea**
Prioridad	Media
Descripción	El sistema debe permitir al usuario ver los datos de una tarea.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una sección para ver los datos de una tarea. 2. Al acceder a la sección, se mostrarán los campos: nombre de la tarea, descripción, fecha de creación, fecha de realización, y los miembros asignados. 3. Si no hay datos disponibles, mostrar un mensaje indicando que no hay datos disponibles.

Cuadro 6.17: RF_17

RF_18	**Ver calendario de tareas**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario ver el calendario de tareas.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una sección para ver el calendario de tareas. 2. Al acceder a la sección, se mostrará un calendario con las tareas asignadas. 3. Al pulsar en una tarea, se mostrará la información detallada de la misma. 4. Si no hay tareas asignadas, mostrar un mensaje indicando que no hay tareas asignadas.

Cuadro 6.18: RF_18

6.1.1.4. Grupo 4: Gestión de gastos

RF_19	**Crear gasto**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario crear un gasto.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para crear un gasto. 2. Al pulsar el botón, se mostrará un formulario con los campos: nombre del gasto, descripción, cantidad y miembros que lo han realizado. 3. Verificar que los campos no estén vacíos. 4. Si los campos no son válidos, mostrar un mensaje de error. 5. Tras la creación del gasto, se redirigirá a la pantalla principal del hogar.

Cuadro 6.19: RF_19

RF_20	Modificar gasto
Prioridad	Alta
Descripción	El sistema debe permitir al usuario modificar un gasto.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para modificar un gasto. 2. Al pulsar el botón, se mostrará un formulario con los campos del gasto a modificar. 3. Verificar que los campos no estén vacíos. 4. Si los campos no son válidos, mostrar un mensaje de error. 5. Tras la modificación del gasto, se redirigirá a la pantalla principal del hogar.

Cuadro 6.20: RF_20

RF_21	Eliminar gasto
Prioridad	Alta
Descripción	El sistema debe permitir al usuario eliminar un gasto.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para eliminar un gasto. 2. Al pulsar el botón, se mostrará un mensaje de confirmación. 3. Si el usuario confirma, el gasto se eliminará y se actualizará la información en la base de datos. 4. Si el usuario cancela, no se realizará ninguna acción.

Cuadro 6.21: RF_21

RF_22	**Saldar deuda**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario saldar una deuda.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para saldar una deuda. 2. Al pulsar el botón, se mostrará un mensaje de confirmación. 3. Si el usuario confirma, la deuda se marcará como saldada y se actualizará la información en la base de datos. 4. Si el usuario cancela, no se realizará ninguna acción.

Cuadro 6.22: RF_22

RF_23	**Ver deudas**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario ver las deudas pendientes.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una sección para ver las deudas pendientes. 2. Al acceder a la sección, se mostrará una lista con las deudas pendientes, incluyendo el nombre del deudor, la cantidad y la fecha. 3. Si no hay deudas pendientes, mostrar un mensaje indicando que no hay deudas pendientes.

Cuadro 6.23: RF_23

RF_24	**Ver detalles de un gasto**
Prioridad	Media
Descripción	El sistema debe permitir al usuario ver los detalles de un gasto.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una sección para ver los detalles de un gasto. 2. Al acceder a la sección, se mostrarán los campos: nombre del gasto, descripción, cantidad, fecha y miembros que lo han realizado. 3. Si no hay datos disponibles, mostrar un mensaje indicando que no hay datos disponibles.

Cuadro 6.24: RF_24

6.1.1.5. Grupo 5: Gestión de compra

RF_25	**Crear lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario crear una lista de la compra.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para crear una lista de la compra. 2. Al pulsar el botón, se mostrará un formulario con los campos: nombre de la lista. 3. Verificar que los campos no estén vacíos. 4. Si los campos no son válidos, mostrar un mensaje de error. 5. Tras la creación de la lista, se redirigirá a la pantalla principal de la lista de la compra.

Cuadro 6.25: RF_25

RF_26	**Añadir producto a la lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario añadir un producto a la lista de la compra.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para añadir un producto a la lista de la compra. 2. Al pulsar el botón, se mostrará un formulario con los campos: nombre del producto y cantidad. 3. Verificar que los campos no estén vacíos. 4. Si los campos no son válidos, mostrar un mensaje de error. 5. Tras añadir el producto, se actualizará la lista de la compra y se redirigirá a la pantalla principal de la lista de la compra.

Cuadro 6.26: RF_26

RF_27	**Eliminar producto de la lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario eliminar un producto de la lista de la compra.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para eliminar un producto de la lista de la compra. 2. Al pulsar el botón, se mostrará un mensaje de confirmación. 3. Si el usuario confirma, el producto se eliminará de la lista de la compra y se actualizará la información en la base de datos. 4. Si el usuario cancela, no se realizará ninguna acción.

Cuadro 6.27: RF_27

RF_28	**Eliminar lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario eliminar una lista de la compra.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para eliminar una lista de la compra. 2. Al pulsar el botón, se mostrará un mensaje de confirmación. 3. Si el usuario confirma, la lista de la compra se eliminará y se actualizará la información en la base de datos. 4. Si el usuario cancela, no se realizará ninguna acción.

Cuadro 6.28: RF_28

RF_29	**Ver productos de la lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario ver los productos de una lista de la compra.
Criterios de aceptación	1. La interfaz de la aplicación debe contener una sección para ver los productos de una lista de la compra. 2. Al acceder a la sección, se mostrará una lista con los productos, incluyendo el nombre del producto y la cantidad. 3. Si no hay productos en la lista, mostrar un mensaje indicando que no hay productos en la lista.

Cuadro 6.29: RF_29

RF_30	**Editar producto de la lista de la compra**
Prioridad	Alta
Descripción	El sistema debe permitir al usuario editar un producto de la lista de la compra.
Criterios de aceptación	1. La interfaz de la aplicación debe contener un botón para editar un producto de la lista de la compra. 2. Al pulsar el botón, se mostrará un formulario con los campos del producto a editar: nombre del producto y cantidad. 3. Verificar que los campos no estén vacíos. 4. Si los campos no son válidos, mostrar un mensaje de error. 5. Tras la edición del producto, se actualizará la lista de la compra y se redirigirá a la pantalla principal de la lista de la compra.

Cuadro 6.30: RF_30

6.1.2. Requisitos no funcionales

Los **requisitos no funcionales** son aquellos que describen las características y restricciones del sistema, pero no están directamentes relacionadas con sus funcionalidades. En lugar de especificar qué hace el sistema, establecen cómo debe hacerlo. Estos requisitos garantizan la calidad, el rendimiento, la seguridad, la usabilidad y otros aspectos clave del software. (Sommerville, 2015).

RNF_01	**Seguridad**
Prioridad	Alta
Descripción	El sistema debe proteger los datos que los usuarios introduzcan.

Cuadro 6.31: RNF_01

RNF_02	**Usabilidad**
Prioridad	Alta
Descripción	El sistema debe ser fácil de usar y comprender para los usuarios.

Cuadro 6.32: RNF_02

RNF_03	**Rendimiento**
Prioridad	Bajo
Descripción	El sistema debe ser rápido y eficiente en la ejecución de las tareas.

Cuadro 6.33: RNF_03

RNF_04	**Compatibilidad**
Prioridad	Media
Descripción	El software debe estar disponible para diferentes sistemas operativos (iOS, Android).

Cuadro 6.34: RNF_04

RNF_05	**Accesibilidad**
Prioridad	Baja
Descripción	El sistema debe ser accesible para usuarios con discapacidades, cumpliendo con las pautas de accesibilidad web.

Cuadro 6.35: RNF_05

6.2. Modelos de casos de uso

Los modelos de de casos de uso nos permiten representar las iteracciones entre los usuarios y el sistema de forma clara. En este apartado, se presentan los casos de uso de la aplicación que reflejan las principales funcionalidades que la aplicación va a ofrecer.

Cada caso de uso describe una funcionalidad específica, estableciendo los actores involucrados. En este caso los actores son:

- **Usuario**: La persona que utiliza la aplicación.
- **Administrador**: La persona que gestiona la aplicación y la base de datos.

6.2.1. Diagrama de casos de uso

A continuación, se muestran los diagramas de casos de uso, que se han dividido en paquetes (cada paquete corresponde a un grupo de funcionales que ya establecimos en el apartado de requisitos funcionales) y así poder representar de forma más clara las funcionalidades de la aplicación.

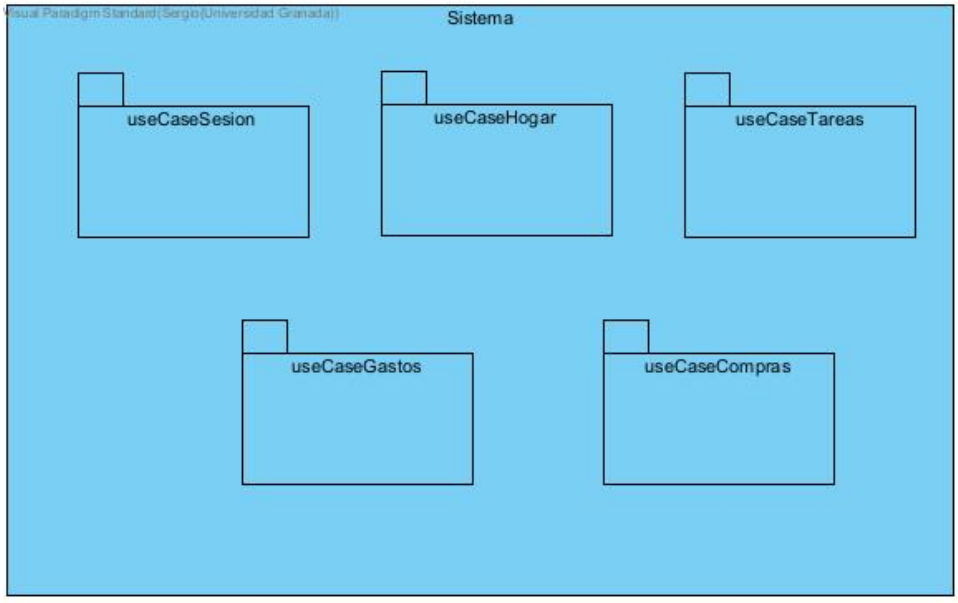


Figura 6.1: Diagrama de paquetes de casos de uso

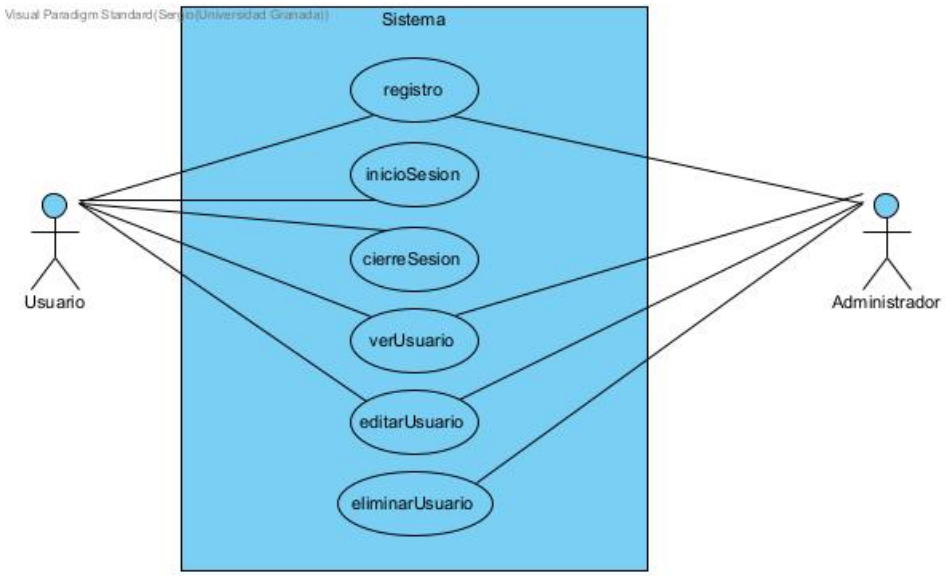


Figura 6.2: Diagrama de casos de uso de la gestión de sesión

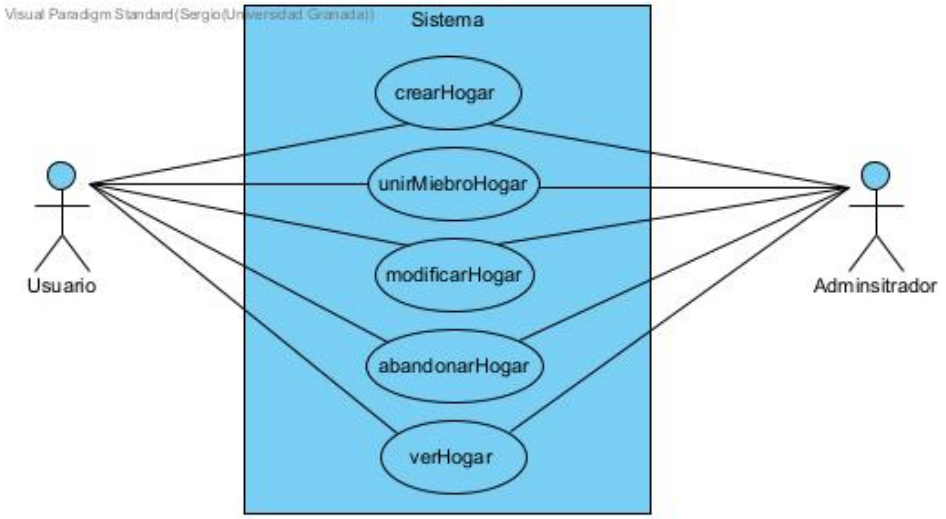


Figura 6.3: Diagrama de casos de uso de la gestion de hogar

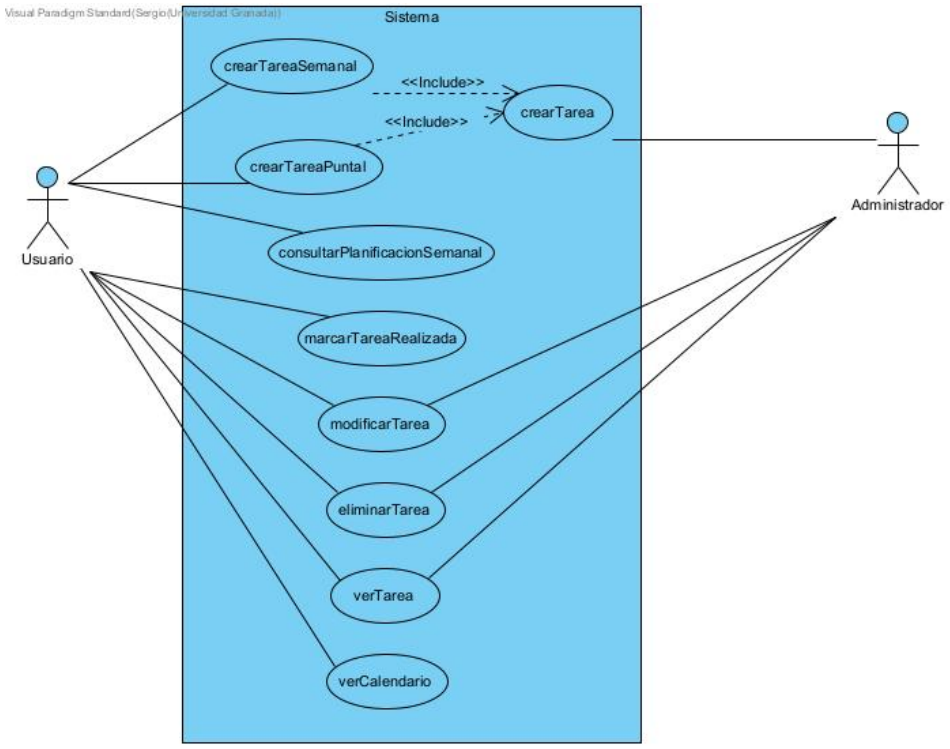


Figura 6.4: Diagrama de casos de uso de la gestion de tareas

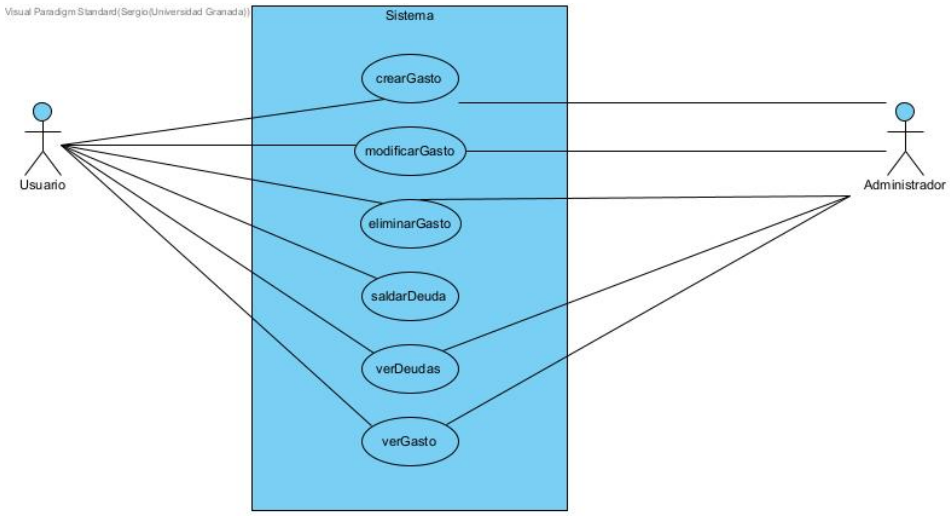


Figura 6.5: Diagrama de casos de uso de la gestion de gastos

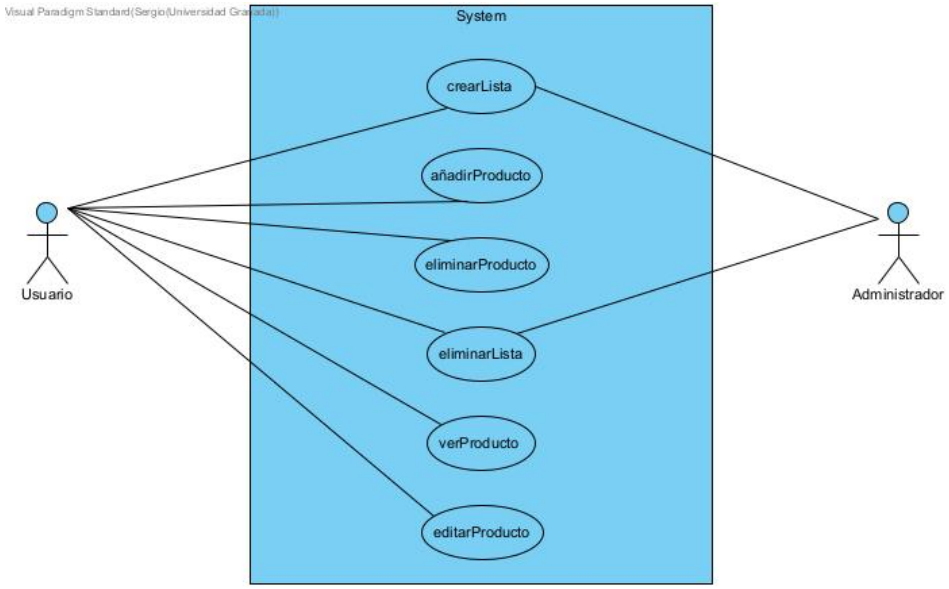


Figura 6.6: Diagrama de casos de uso de la gestion de compra

6.2.2. Descripción de los casos de uso

En esta sección se describen los casos de uso de la aplicación, indicando los actores involucrados, la descripción y los pasos a seguir para llevar a cabo la funcionalidad.

Caso de Uso	registro		CU1
Actores	Usuario, Administrador		
Tipo	Obligatorio		
Referencias	RF_01		
Precondición	El usuario no debe estar registrado en el sistema.		
Postcondición	El usuario queda registrado en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de crearse una cuenta en el sistema.
Resumen	El usuario introduce los datos necesarios para crear una cuenta.

Curso normal			
1.	Usuario: Introduce su nombre, apellidos, correo electrónico y contraseña.	2.	El sistema verifica que los campos no estén vacíos y que la información introducida sea válida.
3.	Usuario: Confirma el registro.	4.	El sistema verifica que el correo electrónico no esté registrado previamente.
5.	Usuario: Recibe un mensaje indicando que debe verificar su correo electrónico.	6.	El sistema redirige al usuario a la pantalla de inicio de sesión.
Cursos alternos			
2a.	Los campos están vacíos o la información no es válida.		
2b.	El correo electrónico ya está registrado.		
2c.	El sistema muestra un mensaje de error.		

Cuadro 6.36: Caso de uso - Registro

Caso de Uso	iniciarSesion		CU2
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_02		
Precondición	El usuario debe estar registrado en el sistema.		
Postcondición	El usuario accede al sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de iniciar sesión en el sistema.
Resumen	El usuario introduce su correo electrónico y contraseña para autenticarse.

Curso normal			
1.	Usuario: Introduce su correo electrónico y contraseña.	2.	El sistema verifica que los campos no estén vacíos y que la información introducida sea válida.
3.	Usuario: Confirma el inicio de sesión.	4.	El sistema verifica que el correo electrónico y la contraseña sean correctos.
5.	Usuario: Accede a la pantalla principal de la aplicación.	6.	El sistema redirige al usuario a la pantalla principal de la aplicación.
Cursos alternos			
2a.	Los campos están vacíos o la información no es válida.		
2b.	El correo electrónico o la contraseña son incorrectos.		
2c.	El sistema muestra un mensaje de error.		

Cuadro 6.37: Caso de uso - Iniciar Sesión

Caso de Uso	cerrarSesion		CU3
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_04		
Precondición	El usuario debe estar autenticado en el sistema.		
Postcondición	El usuario cierra su sesión en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de cerrar su sesión en el sistema.
Resumen	El usuario cierra su sesión para finalizar su uso del sistema.

Curso normal			
1.	Usuario: Pulsa el botón de cerrar sesión.	2.	El sistema muestra un mensaje de confirmación.
3.	Usuario: Confirma que desea cerrar sesión.	4.	El sistema cierra la sesión del usuario y redirige a la pantalla de inicio de sesión.

Cuadro 6.38: Caso de uso - Cerrar Sesión

La explicación de los casos de uso restantes se encuentra en el Apéndice A.

Capítulo 7

Diseño

7.1. Diseño de la arquitectura

7.2. Diseño de la interfaz gráfica

7.2.1. Interfaces de gráficas del Grupo 1: *Gestión de sesión*

En esta sección podemos ver todos los diseños de las interfaces gráficas del Grupo 1: *Gestión de sesión*. En la imagen 7.1 se muestra el diseño de la pantalla de bienvenida, en la imagen 7.2 se muestra el diseño de la pantalla de inicio de sesión que se corresponde con el RF_02 , en la imagen 7.3 se muestra el diseño de la pantalla de registro que se corresponde con el RF_01 , en la imagen 7.4 se muestra el diseño de cierre de sesión y datos de usuario que se corresponde con el RF_04 y RF_05.



Figura 7.1: Diseño de la pantalla de bienvenida.



Figura 7.2: Diseño de la pantalla de inicio de sesión.

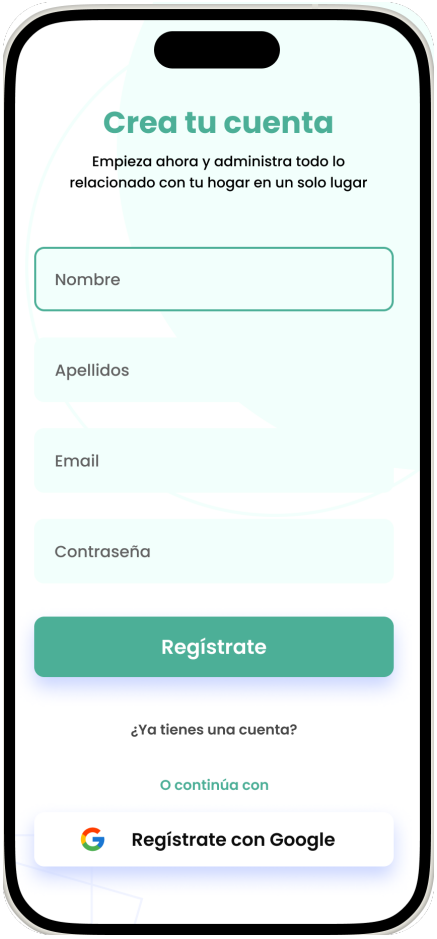


Figura 7.3: Diseño de la pantalla de registro.

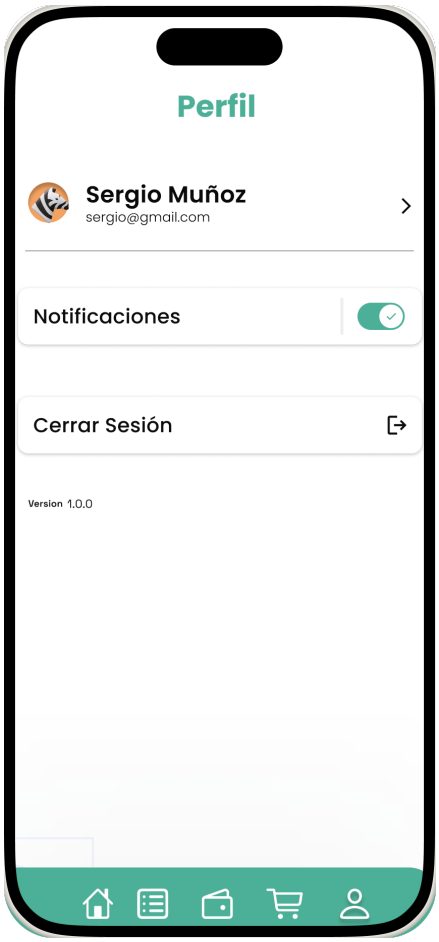


Figura 7.4: Diseño de la pantalla de cierre de sesión y datos de usuario.



Figura 7.5: Diseño de la pantalla de editar perfil.

7.2.2. Interfaces de gráficas del Grupo 2: *Gestión del hogar*



Figura 7.6: Diseño de la pantalla para unirte a un hogar o crear uno nuevo.



Figura 7.7: Diseño de la pantalla para unirte a un hogar.

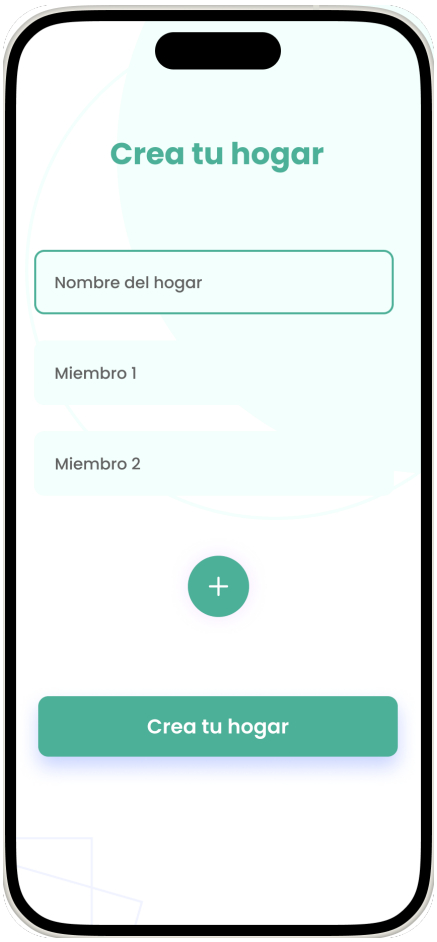


Figura 7.8: Diseño de la pantalla para crear un nuevo hogar.

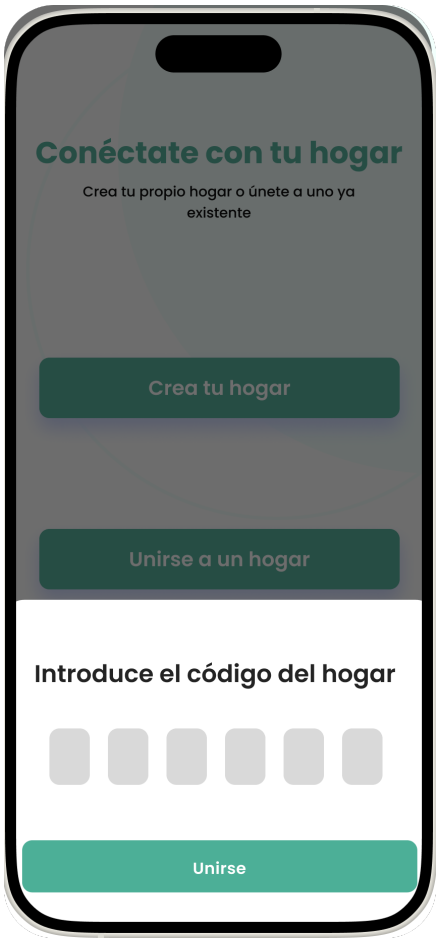


Figura 7.9: Diseño de la pantalla para abandonar a un hogar.

7.2.3. Interfaces de gráficas del Grupo 3: *Gestión de tareas*

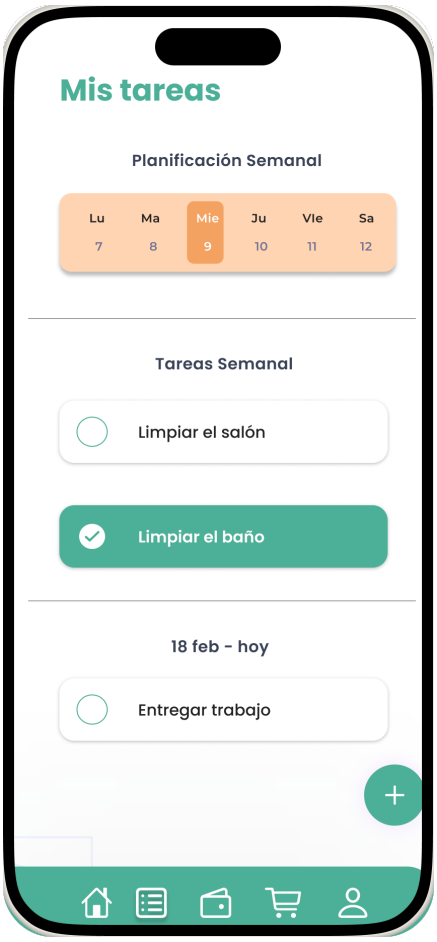


Figura 7.10: Diseño de la pantalla principal de tareas.

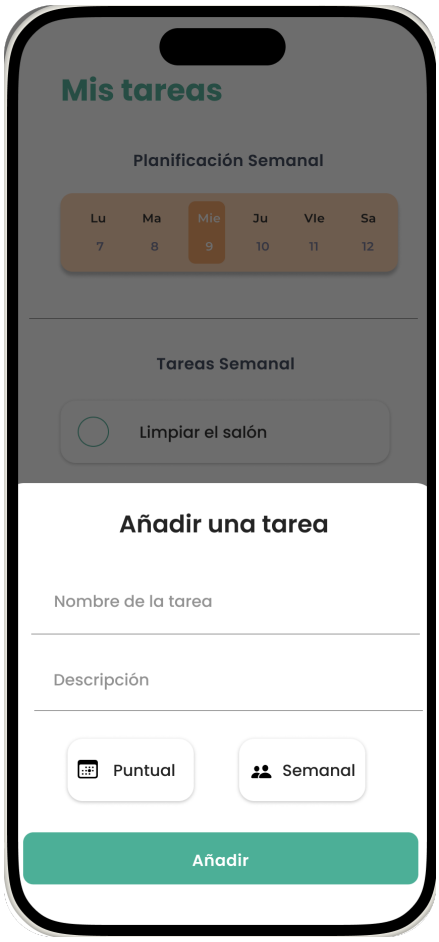


Figura 7.11: Diseño de la pantalla para añadir una tarea.

7.2.4. Interfaces de gráficas del Grupo 4: *Gestión de gastos*

7.2.5. Interfaces de gráficas del Grupo 5: *Gestión de la compra*

Apéndice A

Descripción de casos de uso

Caso de Uso	registro		CU1
Actores	Usuario, Administrador		
Tipo	Obligatorio		
Referencias	RF_01		
Precondición	El usuario no debe estar registrado en el sistema.		
Postcondición	El usuario queda registrado en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de crearse una cuenta en el sistema.
Resumen	El usuario introduce los datos necesarios para crear una cuenta.

Curso normal			
1.	Usuario: Introduce su nombre, apellidos, correo electrónico y contraseña.	2.	El sistema verifica que los campos no estén vacíos y que la información introducida sea válida.
3.	Usuario: Confirma el registro.	4.	El sistema verifica que el correo electrónico no esté registrado previamente.
5.	Usuario: Recibe un mensaje indicando que debe verificar su correo electrónico.	6.	El sistema redirige al usuario a la pantalla de inicio de sesión.
Cursos alternos			
2a.	Los campos están vacíos o la información no es válida.		
2b.	El correo electrónico ya está registrado.		
2c.	El sistema muestra un mensaje de error.		

Cuadro A.1: Caso de uso - Registro

Caso de Uso	iniciarSesion		CU2
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_02		
Precondición	El usuario debe estar registrado en el sistema.		
Postcondición	El usuario accede al sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de iniciar sesión en el sistema.
Resumen	El usuario introduce su correo electrónico y contraseña para autenticarse.

Curso normal			
1.	Usuario: Introduce su correo electrónico y contraseña.	2.	El sistema verifica que los campos no estén vacíos y que la información introducida sea válida.
3.	Usuario: Confirma el inicio de sesión.	4.	El sistema verifica que el correo electrónico y la contraseña sean correctos.
5.	Usuario: Accede a la pantalla principal de la aplicación.	6.	El sistema redirige al usuario a la pantalla principal de la aplicación.
Cursos alternos			
2a.	Los campos están vacíos o la información no es válida.		
2b.	El correo electrónico o la contraseña son incorrectos.		
2c.	El sistema muestra un mensaje de error.		

Cuadro A.2: Caso de uso - Iniciar Sesión

Caso de Uso	cerrarSesion		CU3
Actores	Usuario		
Tipo	Obligatorio		
Referencias	RF_04		
Precondición	El usuario debe estar autenticado en el sistema.		
Postcondición	El usuario cierra su sesión en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de cerrar su sesión en el sistema.
Resumen	El usuario cierra su sesión para finalizar su uso del sistema.

Curso normal			
1.	Usuario: Pulsa el botón de cerrar sesión.	2.	El sistema muestra un mensaje de confirmación.
3.	Usuario: Confirma que desea cerrar sesión.	4.	El sistema cierra la sesión del usuario y redirige a la pantalla de inicio de sesión.

Cuadro A.3: Caso de uso - Cerrar Sesión

Caso de Uso	verUsuario		CU4
Actores	Usuario, Administrador		
Tipo	Obligatorio		
Referencias	RF_05		
Precondición	El usuario debe estar autenticado en el sistema.		
Postcondición	El usuario visualiza la información de su perfil.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de ver la información de su perfil.
Resumen	El usuario accede a la sección de perfil para visualizar su información personal.

Curso normal			
1.	Usuario: Accede a la sección de perfil.	2.	El sistema muestra la información del perfil del usuario.

Cuadro A.4: Caso de uso - Ver Usuario

Caso de Uso	editarUsuario		CU5
Actores	Usuario, Administrador		
Tipo	Obligatorio		
Referencias	RF_06		
Precondición	El usuario debe estar autenticado en el sistema.		
Postcondición	El usuario actualiza su información en el sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El usuario debe ser capaz de editar la información de su perfil.
Resumen	El usuario accede a la sección de perfil y modifica su información personal.

Curso normal			
1.	Usuario: Accede a la sección de perfil.	2.	El sistema muestra la información del perfil del usuario.
3.	Usuario: Modifica la información deseada.	4.	El sistema verifica que la información introducida sea válida.
5.	Usuario: Guarda los cambios.	6.	El sistema actualiza la información del usuario y muestra un mensaje de confirmación.
Cursos alternos			
4a.	La información introducida no es válida.		
4b.	El sistema muestra un mensaje de error.		

Cuadro A.5: Caso de uso - Editar Usuario

Caso de Uso	eliminarUsuario		CU6
Actores	Administrador		
Tipo	Opcional		
Referencias			
Precondición	El usuario debe estar autenticado en el sistema.		
Postcondición	El usuario es eliminado del sistema.		
Autor	Sergio Muñoz		
Fecha	28/02/2025	Versión	1.0

Propósito	El administrador debe ser capaz de eliminar un usuario del sistema.
Resumen	El administrador selecciona un usuario y lo elimina del sistema.

Curso normal			
1.	Administrador: Accede a la lista de usuarios.	2.	El sistema muestra la lista de usuarios registrados.
3.	Administrador: Selecciona el usuario a eliminar.	4.	El sistema muestra un mensaje de confirmación.
5.	Administrador: Confirma la eliminación del usuario.	6.	El sistema elimina al usuario y muestra un mensaje de confirmación.

Cuadro A.6: Caso de uso - Eliminar Usuario

Referencias

Adverthia. (2019). *Usabilidad web: definición y características.* Descargado 2025-02-04, de https://adverthia.com/usabilidad-web-definicion-y-caracteristicas

American Time Use Survey. (2022). *Distributions of time for household management and shared living challenges.* Descargado de https://www.bls.gov/news.release/archives/atus_06222023.pdf

Apple Inc. (2025). *Swift programming language.* Descargado de https://www.swift.org/

Atlassian. (2025). *Guía de procesos iterativos.* Descargado de https://www.atlassian.com/es/work-management/project-management/iterative-process

Beck, K. (1999). *Extreme programming explained: Embrace change.* Addison-Wesley.

Boehm, B. (1988). A spiral model of software development and enhancement. *ACM SIGSOFT Software Engineering Notes*, *11*(4), 14-24.

Burriel, T. (2024). *Accesibilidad en aplicaciones móviles.* Descargado 2025-02-04, de https://torresburriel.com/weblog/accesibilidad-en-aplicaciones-moviles

Cozi Inc. (2025). *Cozi family organizer.* Descargado de https://www.cozi.com/

FC Research. (2024). *Pisos compartidos: Tendencias y perspectivas.* Descargado de https://s36360.pcdn.co/wp-content/uploads/2024/09/NdP-Pisos-compartidos-I-FC-Research.pdf

Foundation, D. S. (2025). *Django - the web framework for perfectionists with deadlines.* Descargado de https://www.djangoproject.com/

Google. (2025a). *Firebase.* Descargado de https://firebase.google.com/

Google. (2025b). *Flutter - build apps for any screen.* Descargado de https://flutter.dev/

Huffington Post. (2025). *El 87 % de los jóvenes emancipados comparte vivienda en alquiler al no poder hacer frente a los gastos mensuales.* Descargado de https://www.huffingtonpost.es/politica/el-87-jovenes-emancipados-comparte-vivienda-alquiler-frente-gastos-mensuales.html

Ionic Framework. (2025). *Ionic framework - the cross-platform app development leader.* Descargado de https://ionicframework.com/

Jefatura del Estado. (2018). *Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales.* Descargado de https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673

JetBrains. (2025). *Kotlin programming language.* Descargado de https://kotlinlang.org/

Kim, G., Humble, J., Debois, P., y Willis, J. (2016). *The devops handbook.* IT Revolution.

Larman, C. (2003). *Agile and iterative development: A manager's guide.* Addison-Wesley.

Meta Platforms, I. (2025). *React native · learn once, write anywhere.* Descargado de https://reactnative.dev/

Microsoft. (2025). *Xamarin.* Descargado de https://dotnet.microsoft.com/apps/xamarin

Ministerio de Igualdad. (2023). *Igualdad presenta 'metoca', la aplicación para repartir de forma corresponsable las tareas domésticas y de cuidados.* Descargado de https://www.igualdad.gob.es/comunicacion/notasprensa/igualdad-presenta-metoca-la-aplicacion-para-repartir-de-forma-corresponsable-las-tareas-domesticas-y-de-cuidados/

Mozilla Developer Network. (2024). *¿qué es la accesibilidad?* Descargado 2025-02-04, de https://developer.mozilla.org/es/docs/Learn_web_development/Core/Accessibility/What_is_accessibility

Nielsen, J. (1993). *Usability engineering* (1st ed.). San Diego: Academic Press.

Node.js Foundation. (2025). *Node.js - javascript runtime.* Descargado de https://nodejs.org/

OurHome. (2025). *Ourhome - organiza tu hogar fácilmente.* Descargado de https://ourhomeapp.com/

Parlamento Europeo. (2016). *Reglamento (ue) 2016/679 del parlamento europeo y del consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos.* Descargado de https://eur-lex.europa.eu/legal-content/ES/TXT/PDF/?uri=CELEX%3A32016R0679

PMO Informática. (2017). *Requerimientos funcionales: definición y ejemplos.* Descargado 2025-02-04, de https://www.pmoinformatica.com/2017/02/requerimientos-funcionales-ejemplos.html

Ruby on Rails. (2025). *Ruby on rails - web application framework.* Descargado de https://rubyonrails.org/

Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson.

Staff, C. (2023). *What is usability and why it matters.* Descargado 2025-02-04, de https://www.coursera.org/mx/articles/what-is-usability-and-why-it-matters

Talent.com. (2025). *Salario de programador junior en españa.* Descargado de https://es.talent.com/salary?job=programador+junior

ThoughtWorks. (2020). *Manual de accesibilidad para profesionales.* Descargado de https://www.thoughtworks.com/content/

dam/thoughtworks/documents/e-book/tw_ebook_accessibility_handbook_v4_spanish.pdf

Todoist. (2025). *Todoist - organiza tu vida y trabajo.* Descargado de https://todoist.com/es

Tricount. (2025). *Tricount - organiza los gastos del grupo.* Descargado de https://tricount.com/es-es

U.S. Department of Housing and Urban Development. (2022). *Desafíos financieros en viviendas estudiantiles fuera del campus: Un análisis de la carga financiera de los arrendamientos anuales y la gestión de los gastos compartidos.* Descargado 2025-02-04, de https://www.huduser.gov/portal/periodicals/insight/insight_2.pdf